

MANUFACTURING

Parts inspection for quality control

Assembly, sorting, dispensing, locating, and packaging of parts

MEDICAL

Screening x-ray, tomographic, ultrasound, and other medical images

DEFENSE

Photo reconnaissance, analysis, and scene interpretation

Target detection, identification, and tracking

Microbe detection and identification

Weapons guidance

Remote and local site monitoring

BUSINESS

Visual document readers

Design tools for engineers and architects

Inspection of labels for identification

Inspection of products for contents and packaging

ROBOTICS

Guidance of welders and spray paint nozzles

Sorting, picking, and bin packing of items

Autonomous guidance of land, air and sea vehicles

SPACE EXPLORATION

Discovery and interpretation of astronomical images

Terrestrial image mapping and interpretation for plant disease, mineral deposits, insect infestations, and soil erosion

Vision in an organic system is the process of sensing a pattern of light energy, and developing an interpretation of those patterns. The sensing part of the process consists of selectively gathering light from some area of the environment, focusing and projecting it onto a light sensitive surface, and converting the light into electrochemical patterns of impulses. The perception part of the process involves the transformation and comparison of the transmitted impulse patterns to other prestored patterns

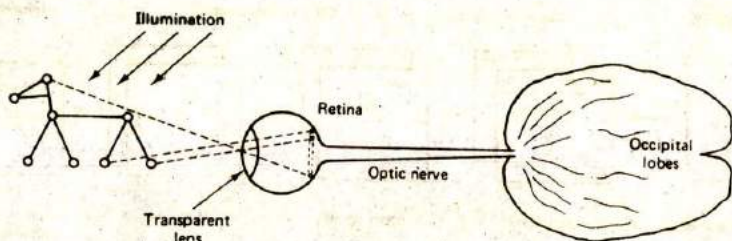


Figure 14.1 The human process of visual interpretation.

together with some form of inference. The basic vision process as it occurs in humans is depicted in Figure 14.1.

Light from illuminated objects is collected by the transparent lens of the eye, focused, and projected onto the retina where some 250 million light sensitive sensors (cones and rods) are excited. When excited, the sensors send impulses through the optic nerve to the visual cortex of the occipital lobes of the brain where the images are interpreted and recognized.

Computer vision systems share some similarities with human visual systems, at least as we now understand them. They also have a number of important differences. Although artificial vision systems vary widely with the specific application, we adopt a general approach here, one in which the ultimate objective is to determine a high-level description of a three-dimensional scene with a competency level comparable to that of human vision systems. Before proceeding farther we should distinguish between a scene and an image of a scene. A scene is the set of physical objects in a picture area, whereas an image is the projection of the scene onto a two-dimensional plane.

With the above objectives in mind, a typical computer vision system should be able to perform the following operations:

1. Image formation, sensing, and digitization
2. Local processing and image segmentation
3. Shape formation and interpretation
4. Semantic analysis and description.

The sequence of these operations is depicted in Figure 14.2.

As we proceed through the processing stages of computer vision, the reader will no doubt be impressed by the similarities and parallels one can draw between vision processing and natural language processing. The image-sensor stage in vision corresponds to speech recognition in language understanding, the low and intermediate processing levels of vision correspond to syntactic and semantic language processing respectively, and high level processing, in both cases, corresponds to the process of building and interpreting high level knowledge structures.

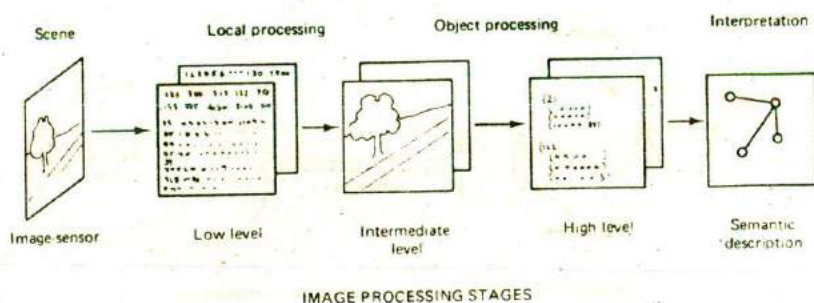


Figure 14.2 Processing stages in computer vision systems.

Vision Processing Overview

The input to a vision system is a two dimensional image collected on some form of light sensitive surface. This surface is scanned by some means to produce a continuous voltage output that is proportional to the light intensity of the image on the surface. The output voltage $f(x, y)$ is sampled at a discrete number of x and y points or pixel (picture element) positions and converted to numbers. The numbers correspond to the gray level intensity for black and white images. For color images, the intensity value is comprised of three separate arrays of numbers, one for the intensity value of each of the basic colors (red, green, and blue).

Thus, through the digitization process, the image is transformed from a continuous light source into an array of numbers which correspond to the local image intensities at the corresponding x - y pixel positions on the light sensitive surface.

Using the array of numbers, certain low level operations are performed, such as smoothing of neighboring points to reduce noise, finding outlines of objects or edge elements, thresholding (recording maximum and minimum values only, depending on some fixed intensity threshold level), and determining texture, color, and other object features. These initial processing steps are ones which are used to locate and accentuate object boundaries and other structure within the image.

The next stage of processing, the intermediate level, involves connecting, filling in, and combining boundaries, determining regions, and assigning descriptive labels to objects that have been accentuated in the first stage. This stage builds higher level structures from the lower level elements of the first stage. When complete, it passes on labeled surfaces such as geometrical objects that may be capable of identification.

High-level image processing consists of identifying the important objects in the image and their relationships for subsequent description as well-defined knowledge structures and hence, for use by a reasoning component.

Special types of vision systems may also require three dimensional processing and analysis as well as motion detection and analysis.

The Objectives of Computer Vision Systems

The ultimate goals of computer image understanding is to build systems that equal or exceed the capabilities of human vision systems. Ideally, a computer vision system would be capable of interpreting and describing any complex scene in complete detail. This means that the system must not only be able to identify a myriad of complex objects, but must also be able to reason about the objects, to describe their function and purpose, what has taken place in the scene, why any visible or implied events occurred, what is likely to happen, and what the objects in the scene are capable of doing.

Figure 14.3 presents an example of a complex scene that humans can interpret well with little effort. It is the objective of many researchers in computer vision to build systems capable of interpreting, describing, and reasoning about scenes of this type in real time. Unfortunately, we are far from achieving this level of competency. To be sure, some interesting vision systems have been developed, but they are quite crude compared to the elegant vision systems of humans.

Like natural language understanding, computer vision interpretation is a difficult problem. The amount of processing and storage required to interpret and describe a complex scene can be enormous. For example, a single image for a high resolution aerial photograph may result in some four to nine million pixels (bytes) of information and require on the average some 10 to 20 computations per pixel. Thus, when several frames must be stored during processing, as many as 100 megabytes of storage may be needed, and more than 100 million computations performed.

THE FAR SIDE

By GARY LARSON

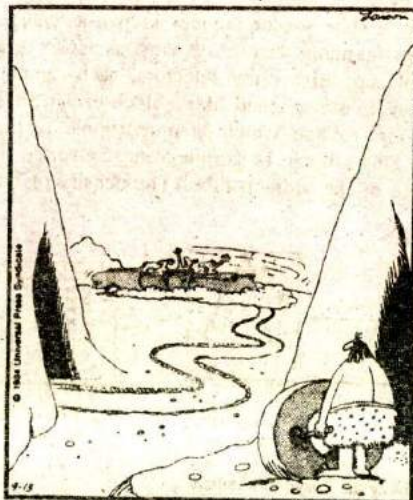


Figure 14.3 Example of a complex scene.
(THE FAR SIDE COPYRIGHT 1984
UNIVERSAL PRESS SYNDICATE.
Reprinted by permission.
All rights reserved.)

14.2 IMAGE TRANSFORMATION AND LOW-LEVEL PROCESSING

In this section, we examine the first stages of processing. This includes the process of forming an image and transforming it to an array of numbers which can then be operated on by a computer. In this first stage, only local processing is performed on the numbers to reduce noise and other unwanted picture elements, and to accentuate object boundaries.

Transforming Light Energy to Numbers

The first step in image processing requires a transformation of light energy to numbers, the language of computers. To accomplish this, some form of light sensitive transducer is used such as a vidicon tube or charge-coupled device (CCD).

A vidicon tube is the type of sensor typically found in home or industrial video systems. A lens is used to project the image onto a flat surface of the vidicon. The tube surface is coated with a photoconductive material whose resistance is inversely proportional to the light intensity falling on it. An electron gun is used to produce a flying-spot scanner with which to rapidly scan the surface left to right and top to bottom. The scan results in a time varying voltage which is proportional to the scan spot image intensity. The continuously varying output voltage is then fed to an analog-to-digital converter (ADC) where the voltage amplitude is periodically sampled and converted to numbers. A typical ADC unit will produce 30 complete digitized frames consisting of 256×256 , or 512×512 (or more) samples of an image per second. Each sample is a number (or triple of numbers in the case of color systems) ranging from 0 to 64 (six bits) or 0 to 255 (eight bits). The image conversion process is depicted in Figure 14.4.

A CCD is typical of the class of solid state sensor devices known as charge transfer devices that are now being used in many vision systems. A CCD is a rectangular chip consisting of an array of capacitive photodetectors, each capable of storing an electrostatic charge. The charges are scanned like a clock-driven shift register and converted into a time varying voltage which is proportional to the incident light intensity on the detectors. This voltage is sampled and converted to integers using an ADC unit as in the case of the vidicon tube. The density of the

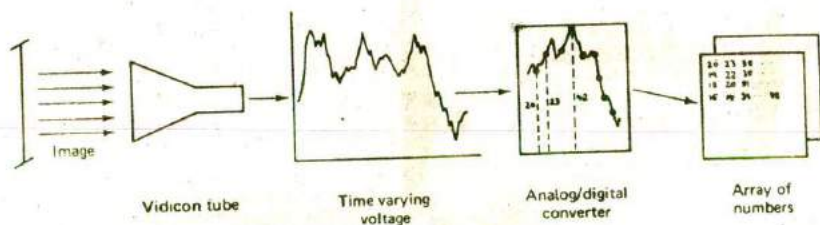


Figure 14.4 Transforming the image to numbers.

detectors on the chip is quite high. For example, a CCD chip of about five square centimeters in area may contain as many as 1000 by 1000 detectors.

The numeric outputs from the ADC units are collected as arrays of numbers which correspond to the light intensity of the image on the surface of the transducer. This is the input to the next stage of processing illustrated in Figure 14.4.

Processing the Quantized Arrays

The array of numbers produced from the image sensing device may be thought of as the lowest, most primitive level of abstraction in the vision understanding process. The next step in the processing hierarchy is to find some structure among the pixels such as pixel clusters which define object boundaries or regions within the image. Thus, it is necessary to transform the array of raw pixel data into regions of discontinuities and homogeneity, to find edges and other delimiters of these object regions.

A raw digitized image will contain some noise and distortion. Therefore, computations to reduce these effects may be necessary before locating edges and regions. Depending on the particular application, low level processing will often require local smoothing of the array to eliminate this noise. Other low level operations include threshold processing to help define homogeneous regions, and different forms of edge detection to define boundaries. We examine some of these low level methods next.

Thresholding is the process of transforming a gray level representation to a binary representation of the image. All digitized array values above some threshold level T are set equal to the maximum gray-level value (black), and values less than or equal to T are set equal to zero (white). For simplicity, assume gray-level values have been normalized to range between zero and one, and suppose a threshold level of $T = 0.7$ has been chosen. Then all array values $g(x,y) > 0.7$ are set equal to 1 and values $g(x,y) \leq 0.7$ are set equal to 0. The result is an array of binary 0 and 1 values. An example of an image that has been thresholded at 0.7 to produce a binary image is illustrated in Figure 14.5.

Thresholding is one way to segment the image into sharpen object regions by enhancing some portions and reducing others like noise and other unwanted features. Thresholding can also help to simplify subsequent processing steps. And in many cases, the use of several different threshold levels may be necessary since low intensity object surfaces will be lost to high threshold levels, and unwanted background will be picked up and enhanced by low threshold levels. Thresholding at several levels may be the best way to determine different regions in the image when it is necessary to compensate for variations in illumination or poor contrast.

Selecting one or more appropriate threshold level settings T_i will require additional computations, such as first producing a histogram of the image gray-level intensities. A histogram gives the frequencies of occurrence of different intensity (or some other feature) levels within the image. An analysis of a histogram can reveal where concentrations of different intensity levels occur, where peaks and broad flat levels occur and where abrupt differences in level occur. From this informa-



Figure 14.5 Threshold transformation of an image.¹

tion the best choice of T_i values are often made apparent. For example, a histogram with two or more clear separations between intensity levels that have a relatively high frequency of occurrence will usually suggest the best threshold levels for object identification and separation. This is seen in Figure 14.6.

Next, we turn to the question of image smoothing. *Smoothing* is a form of digital filtering. It is used to reduce noise and other unwanted features and to enhance certain image features. Smoothing is a form of image transformation that tends to eliminate spikes and flatten widely fluctuating intensity values. Various forms of smoothing techniques have been employed, including local averaging, the use of models, and parametric form fitting.

One common method of smoothing is to replace each pixel in an array with a weighted average of the pixel and its neighboring values. This can be accomplished with the use of filter masks which use some configuration of neighboring pixel values to compute a smoothed replacement value. Two typical masks consist of either four or eight neighboring pixels whose intensity values are used in the weighting

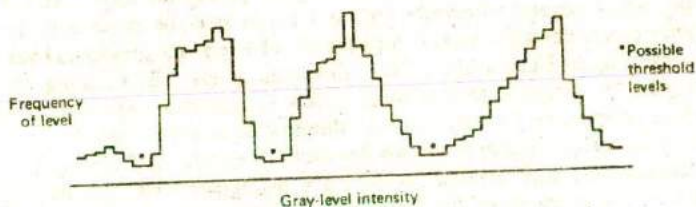


Figure 14.6 Histogram of light intensity levels.

¹ Courtesy of Kenneth Chapman and INTELEDEX, INC.

computation. If smoothing is being performed at pixel location (x, y) , the neighboring pixels are at the eight locations: $(x + 1, y - 1)$, $(x + 1, y)$, $(x + 1, y + 1)$, $(x, y + 1)$, $(x, y - 1)$, $(x - 1, y - 1)$, $(x - 1, y)$, and $(x - 1, y + 1)$. From these, either the four immediate neighbors (top, bottom, left, and right) or all eight neighbors are sometimes chosen.

Examples of smoothing masks for four and eight neighborhood pixels are as follows:

$$\begin{bmatrix} \underline{1/32} & 3/32 & 1/32 \\ 3/32 & \underline{1/2} & 3/32 \\ 1/32 & 3/32 & \underline{1/32} \end{bmatrix} \quad \begin{bmatrix} & 1/8 & \\ 1/8 & \underline{1/2} & 1/8 \\ & 1/8 & \end{bmatrix}$$

The underlined number in each array identifies the pixel being smoothed. (Note that the filter weights in a mask should sum to one to avoid distortions.) Applying a mask to an image array has the effect of reducing spurious noise as well as sharp boundaries. It reduces sharp spikes but also tends to blur the image. For example, when the eight mask filter given above is applied to the array shown in Figure 14.7, the blurring effects are quite pronounced.

The degree of smoothing and hence blurring can, of course, be controlled with the use of appropriate weighting values in the mask. Weighted smoothing of this type over a region is known as convolution. Convolution is sometimes used to smooth an image prior to the application of differential operators which detect edges. We return to the subject of convolution smoothing after we look at the edge detection problem.

Local edge detection is the process of finding a boundary or delimiter between two regions. An edge will show up as a relatively thin line or arc which appears as a measurable difference in contrast between two otherwise homogeneous regions.



Original image array
(a)



Smoothed image array
(b)

Figure 14.7 Application of a smoothing mask.²

²Courtesy of Kenneth Chapman and INTELEDEX, INC.

Regions belonging to the same object are usually distinguishable by one or more features which are relatively homogeneous throughout, such as color, texture, three-dimensional flow effects, or intensity.

Boundaries which separate adjoining regions represent a discontinuity in one or more of these features, a fact that can be exploited by measuring the rate of change of a feature value over the image surface. For example, the rate of change or gradient in intensity in the horizontal and vertical directions can be measured with difference functions D_x and D_y defined as

$$D_x = f(x, y) - f(x - n, y)$$

$$D_y = f(x, y) - f(x, y - n)$$

where n is a small integer greater than or equal to 1.

When an image is scanned horizontally or vertically, D_x and D_y will vary little over homogeneous regions, and show a sharp increase or decrease at locations where discontinuities occur. They are the discrete equivalents of the continuous differential operators used in the calculus. The rate of change of the gradient can also be useful in finding local edges as we will see below. For discrete functions, second order difference operators provide the rate of change of gradient, comparable to second order differential operators.

Since we are interested in locating edges with any given orientation, a better gradient measure is one which is sensitive to intensity changes in any direction. We can achieve this with a directional norm of D_x and D_y such as the vector gradient.

$$D_{xy} = (D_x^2 + D_y^2)^{1/2}$$

$$\theta_{xy} = \tan^{-1}(D_y/D_x)$$

For $n = 1$, D_x and D_y are most easily computed by application of the equivalent weighting masks; the two element masks are $\begin{pmatrix} -1 & 1 \end{pmatrix}$ and $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$ respectively.

An example of the application of these two masks to an image array is illustrated in Figure 14.8 where a vertical edge is seen to be quite pronounced. Masks such as these have been generalized to measure gradients over wider regions covering several pixels. This has the effect of reducing spurious noise and other sharp spikes.

Two masks deserving particular attention are the Prewitt (1970) and Sobel (1970) masks as depicted in Figure 14.9. These masks are used to compute a broadened normalized gradient than the simple masks given above. We leave the details of the computations as one of the exercises at the end of this chapter.

We return now to the methods of edge detection which employ smoothing followed by an application of the gradient. For this, the continuous case is considered first.

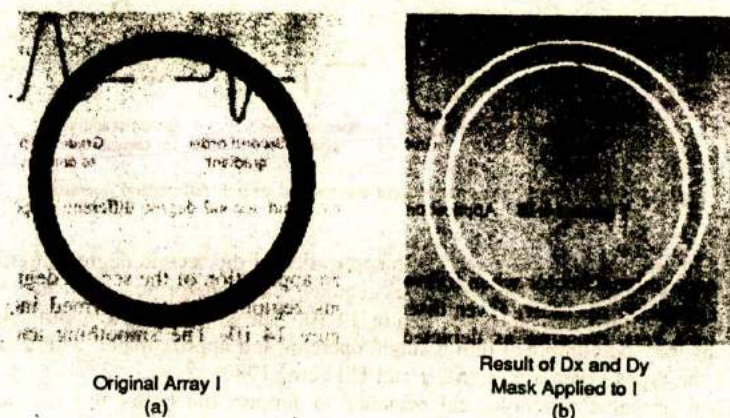


Figure 14.8 Application of difference functions to an image.

The continuous analog of discrete smoothing in one dimension is the convolution of two functions f and g (written $f * g$) where

$$h(y) = f * g = \int f(x)g(y - x)dx$$

Convolving the two functions f and g is similar to computing the cross correlation, a process that reduces random noise and enhances coherent or structural changes.

One particular form of weighting function g has a symmetric bell shape or normal form, that is the Gaussian distribution. The two dimensional form of this function is given by

$$g(u, v) = ce^{-(u^2 + v^2)/2}$$

where c is a normalizing constant.

Because of their rotational symmetry, Gaussian filters produce desirable effects

$P_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$
$P_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$	$S_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$
Prewitt Masks	Sobel Masks

Figure 14.9 Generalized edge detection masks.

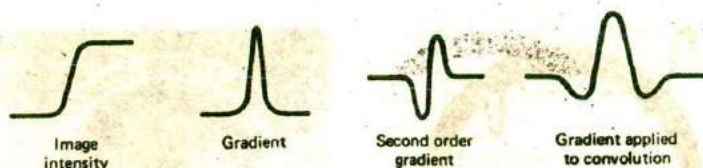


Figure 14.10 Application of Gaussian and second degree differential operators.

as an edge detector when followed by an application of the second degree differential (gradient) operator. Over discontinuous regions, the transformed intensity results in a zero crossing as depicted in Figure 14.10. The smoothing and differencing operations may be combined into a single operator and approximated with a digital mask of the types given above (Marr and Hildreth, 1980).

There is some psychological evidence to support the belief that the human eye uses a form of Gaussian transformation called lateral inhibition which has the effect of enhancing the contrast between gradually changing objects, such as an object and its background.

Another approach used to filter the digitized image applies frequency domain transforms such as the Fourier transform. Since edges represent higher frequency components, the transformed image can be analyzed on the basis of its frequency distribution. For this, the Fourier transform has become one of the most popular transform methods, since an efficient computation algorithm has been developed. It is known as the Fast Fourier transform. The discrete two-dimensional version of the Fourier transform is given by

$$F(u,v) = \frac{1}{n} \sum_{x=0}^{n-1} \sum_{y=0}^{n-1} f(x,y) e^{-2i(xu+yv)/n}$$

Applying this transform to an array of intensity values produces an array of complex numbers that correspond to the spatial frequency components of the image (sums of sine and cosine terms). The transformed array will contain all of the information in the original intensity image, but in a form that is more easily used to identify regions that contain different frequency components. Filtering with the Fourier transform is accomplished by setting the high (or low) values of u and v equal to zero. For example, the value $F(v,v) = F(0,0)$ corresponds to the zero frequency or the DC component, and higher values of u and v correspond to the high frequency components. As with intensity image arrays, thresholding of transformed arrays can be used to separate different frequency components.

The original intensity image with any modifications, is recovered with the inverse transform given by

$$f(x,y) = \frac{1}{n} \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} F(u,v) \exp \left[\frac{2i}{n} (xu + yv) \right]$$

Another method of edge detection is model fitting. This is accomplished by locally fitting a parametric profile of edges to the image array. A model in the form of a mask is shifted over a region and compared to the corresponding gray levels. If the fit between the model and the gray-level pattern scores high enough, an edge with the given orientation is labeled appropriately. Model fitting methods usually require heavy computations. We omit the details here.

Texture and Color

As suggested earlier, texture and color are also used to identify regions and boundaries. Texture is a repeated pattern of elementary shapes occurring on an object's surface. Texture may appear to be regular and periodic, random, or partially periodic. Figure 14.11 illustrates some examples of textured surfaces.

The structure in texture is usually too fine to be resolved, yet still coarse enough to cause noticeable variation in the gray levels. Even so, methods of analysis for texture have been developed. They are commonly based on statistical analyses of small groups of pixels, the application of pattern matching, the use of Fourier transforms, or modeling with special functions known as fractals. These methods are beyond the scope of our goals in this chapter.

The use of color to identify and interpret regions requires more than three times as much processing as gray-level processing. First, the image must be separated into its three primary colors with red, green, and blue filters (Figure 14.12).

The separate color images must then be processed by sampling the intensities and producing three arrays or a single array of tristimulus values. The arrays are then processed separately (in some cases jointly) to determine common color regions and corresponding boundaries. The processes used to find boundaries and regions, and to interpret color images is similar to that of gray-level systems.

Although the additional computation required in color analysis can be significant, the added information gained from separate color intensity arrays may be warranted, depending on the application. In complex scene analysis, color may be the most effective method of segmentation and object identification. In Section 14.6 we describe



Figure 14.11 Examples of textured surfaces.

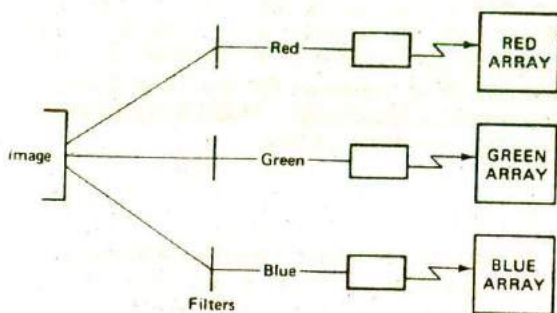


Figure 14.12 Color separation and processing.

an interesting color scene analyser which is based on a rule based inferencing system (Ohta, 1985).

Stereo and Optic Flow

A stereoscopic vision system requires two displaced sensors to obtain two views of objects from different perspectives. The differences between the views makes it possible to estimate distances and derive a three-dimensional model of a scene. The displacement of a pixel from one image to a different location in another image is known as the disparity. It is the disparity between the two views that permit the estimation of the distance to objects in the scene. The human vision system is somehow able to relate the two different images and form a correspondence that translates to a three-dimensional interpretation. Figure 14.13 illustrates the geometric relationships used to estimate distances to objects in stereoscopic systems.

The distance k from the lens to the object can be estimated from the relationships that hold between the sides of the similar triangles. Using the relations $i_1 / e_1 = f / k$, $i_2 / e_2 = f / k$, and $d = e_1 + e_2$ we can write

$$k = fd / (i_1 + i_2)$$

Since f and d are relatively constant, the distance k is a function of the disparity, or sum of the distances i_1 and i_2 .

In computer vision systems, determining the required correspondence between the two displaced images is perhaps the most difficult part in determining the disparity.

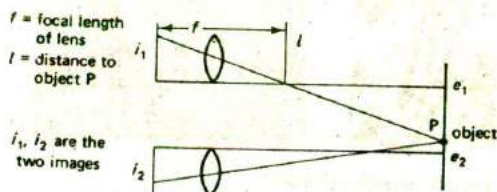


Figure 14.13 Disparity in stereoscopic systems.

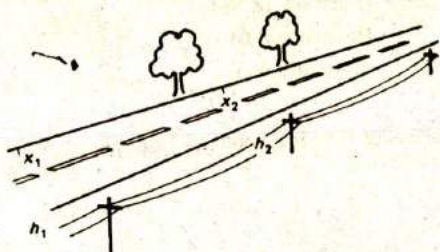


Figure 14.14 Example of optical flow in a scene.

Corresponding pixel groupings in the two images must be located to determine the disparity from which the distance can be estimated. In practice, methods based on correlation, gray-level matching, template matching, and edge contour comparisons have been used to estimate the disparity between stereo images.

Optic flow is an alternative approach to three-dimensional scene analysis which is based on the relative motion of a sensor and objects in the scene. If a sensor is moving (or objects are moving past a sensor), the apparent continuous flow of the objects relative to the sensor is known as optical flow. Distances can be estimated from the change in flow or relative velocity of the sensor and the objects. For example, in Figure 14.14 if the velocity of the sensor is constant, the change in distance dx between points x_1 and x_2 is proportional to the change in size of the power lines h , through the relation

$$dx / dt = k(dh / dt)$$

This relationship is equivalent to the change in size of regular flowing objects with distance from the observer such as highways, railroad tracks, or power lines, as depicted in Figure 14.14.

14.3 INTERMEDIATE-LEVEL IMAGE PROCESSING

The next major level of analysis builds on the low-level or early processing steps described above. It concentrates on segmenting the image space into larger global structures using homogeneous features in pixel regions and boundaries formed from pieces of edges discovered during the low-level processing. This level requires that pieces of edges be combined into contiguous contours which form the outline of objects, partitioning the image into coherent regions, developing models of the segmented objects, and then assigning labels which characterize the object regions.

One way to begin defining a set of objects is to draw a silhouette or sketch of their outlines. Such a sketch has been called the raw primal sketch by Marr (1982). It requires connecting up pieces of edges which have a high likelihood of forming a continuous boundary. For example, the problem is to decide whether two edge pieces such as

- (edge (location 21 103)
(intensity 0.8)
(direction 46))
- (edge (location 18 98)
(intensity 0.6)
(direction 41))

should be connected. This general process of forming contours from pieces of edges is called *segmentation*.

Graphical Edge Finding

Graphical methods can be used to link up pieces of edges. One approach is to use a minimum spanning tree (MST). Starting at any cluster of pixels known to be part of an edge, this method performs a search in the neighborhood of the cluster for groupings with similar feature values. Each such grouping corresponds to a node in an edge tree. When a number of such nodes have been found they are connected using the MST algorithm.

An MST is found by connecting an arc between the first node selected and its closest neighbor node and labeling the two nodes accordingly. Neighborhoods of both connected nodes are then searched. Any node found closest to either of the two connected nodes (below some threshold distance) is then used to form the next branch in the tree. A second arc is constructed between the newly found node and the closest connected node, again labeling the new node. This process is repeated until all nodes having arc distances less than some value (such as a function of the average arc distances) have been connected. An example of an MST is given in Figure 14.15.

Another graphical approach is based on the assignment of a cost or other measure of merit to pixel groupings. The cost assignment can be based on a simple function of features, such as intensity, orientation, or color. A best-first (branch-and-bound) or other form of graph search is then performed using some heuristic function to determine a least-cost path which represents the edge contour.

Other edge finding approaches are based on fitting a low degree polynomial to a number of edge pieces which have been found through local searches. The resultant polynomial curve segment is then taken as the edge boundary. This approach is similar to one which compares edge templates to short groupings of pieces. If a particular matching template scores above some threshold, the template pattern is then used to define the contour.

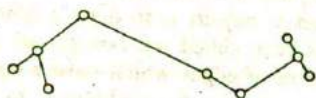


Figure 14.15 Edge finding using a minimum spanning tree.

Edge Finding with Dynamic Programming

Edge following can also be formulated as a dynamic programming problem since picking a best edge path is a type of sequential optimization problem. Candidate edge pieces are assigned a local cost value based on some feature such as intensity, and the path having minimum cost is defined as the edge contour.

Assume that a starting point E_s has been selected. Dynamic programming begins with a portion of the problem and finds an optimal solution for this subproblem. The subproblem is then enlarged, and an optimal solution is found for the enlarged problem. This process continues step-by-step until a global optimum for the whole problem has been found (a path to the terminal edge point E_t).

The process can be described mathematically as a recursive process. Let $C_n(s, t_n)$ be the total cost of the best path for the remaining path increments, given the search is at position (state) s and ready to select t_n as the next move direction. Let t_n^* be the value of t_n that minimizes C_n , and C_n^* the corresponding minimum of C_n . Thus, at each stage, the following values are computed:

$$C_n^*(s) = \min_{t_n} C_n(s, t_n) = C_n(s, t_n^*)$$

where

$$\begin{aligned} C_n(s, t_n) &= (\text{cost at stage } n) \\ &\quad + (\text{minimum costs for stages } n + 1 \text{ onwards}) \\ &= K(s, t_n) + C_{n+1}^*(t_n) \end{aligned}$$

where $K(s, t_n)$ is the cost at stage n , and $C_{n+1}^*(t_n)$ is the minimum cost for stages $n + 1$ to the terminal stage.

The computation process is best understood through an example. Consider the following 5×5 array of pixel cost values.

$$\begin{bmatrix} 9 & 7 & 6 & 5 & 1 \\ 3 & 7 & 2 & 7 & 1 \\ 4 & 1 & 5 & 2 & 7 \\ 6 & 6 & 3 & 7 & 7 \\ 8 & 7 & 2 & 2 & 3 \end{bmatrix}$$

Suppose we wish to find the optimal cost path from the lower left to the upper right corner of the array. We could work from either direction, but we arbitrarily choose to work forward from the lower left pixel with cost value 8. We first set all values except 8 equal to some very large number, say M , and compute the minimum cost of moving from the position with the 8 to all other pixels in the bottom row by adding the cost of moving from pixel to neighboring pixel. This results in the following cost array.

$$\begin{bmatrix} M & M & M & M & M \\ M & M & M & M & M \\ M & M & M & M & M \\ M & M & M & M & M \\ 8 & 15 & 17 & 19 & 22 \end{bmatrix}$$

Next, we compute the minimum neighbor path cost for the next to the last row to obtain

$$\begin{bmatrix} M & M & M & M & M \\ M & M & M & M & M \\ M & M & M & M & M \\ 14 & 14 & 17 & 24 & 29 \\ 8 & 15 & 17 & 19 & 22 \end{bmatrix}$$

Note that the minimum cost path to the second, third and fourth positions in this row is the diagonal path (position 5,1 to 4,2) followed by a horizontal right traversal in the same row, whereas the minimum cost path for the last position in this row is the path passing through the rightmost position of the bottom row. The remaining minimum path costs are computed in a similar fashion, row by row, to obtain the final cost array.

$$\begin{bmatrix} 27 & 24 & 23 & 22 & 21 \\ 18 & 22 & 17 & 24 & 20 \\ 18 & 15 & 19 & 19 & 26 \\ 14 & 14 & 17 & 24 & 29 \\ 8 & 15 & 17 & 19 & 22 \end{bmatrix}$$

From this final minimum cost array, the least cost path is easily found to be

$$(5,1) \rightarrow (4,1) \text{ or } (4,2) \rightarrow (3,2) \rightarrow (2,3) \rightarrow (1,4) \rightarrow (1,5)$$

as depicted with the double line path.

$$\begin{bmatrix} 27 & 24 & 23 & 22=21 \\ 18 & 22 & 17 & 24 & 20 \\ 18 & 15 & 19 & 19 & 26 \\ 14 & 14 & 17 & 24 & 29 \\ 8 & 15 & 17 & 19 & 22 \end{bmatrix}$$

Dynamic programming methods usually result in considerable savings over exhaustive search methods which require an exponential number of computations and comparisons (see Chapter 9), whereas the number of dynamic programming computations for the same size of problem is of linear order.

Region Segmentation through Splitting and Merging

Rather than defining regions with edges, it is possible to build them. For example, global structures can be constructed from groups of pixels by locating, connecting, and defining regions having homogeneous features such as color, texture, or intensity. The resulting segmented regions are expected to correspond to surfaces of objects in the real world. Such coherent regions do not always correspond to meaningful regions, but they do offer another viable approach to the segmentation of an image. When these methods are combined with other segmentation techniques, the confidence level that the regions represent meaningful objects will be high.

Once an image has been segmented into disjointed object areas, the areas can be labeled with their properties and their relationships to other objects, and then identified through model matching or description satisfaction.

Region segmentation may be accomplished by region splitting, by region growing (also called region merging), or by a combination of the two. When splitting is used, the process proceeds in a top-down manner. The image is split successively into smaller and smaller homogeneous pieces until some criteria are satisfied. When growing regions, the process proceeds in a bottom-up fashion. Individual pixels or small groups of pixels are successively merged into contiguous, homogeneous areas. A combined splitting-growing approach will use both bottom-up and top-down techniques.

Regions are usually assumed to be disjointed entities which partition the image such that (1) a given pixel can appear in a single region only, (2) subregions are composed of connected pixels, (3) different regions are disjoint areas, and (4) the complete image area is given by the union of all regions. Regions are usually defined by some homogeneous property such that all pixels belonging to the region satisfy the property, and pixels not satisfying the property lie in a different region. Note that a region need not consist of contiguous pixels only since some objects may be split or covered by occluding surfaces. Condition 2 is needed to insure that all regions are accounted for and that they fill up the complete image area.

In region splitting, the process begins with an entire image which is successively divided into smaller regions which exhibit some coherence in features. One effective method is to apply multiple thresholding levels which can isolate regions having homogeneous features. Histograms are first obtained to establish the threshold levels. This may require masking portions of the image to achieve effective separation of complex objects. Each threshold level can then produce a binary image consisting of all of the objects which exceed the thresholded level. Once the binary regions are formed, they are easily delineated, separated, and marked for subsequent processing. This whole process of masking, computing, and analyzing a histogram, thresholding, defining an area, masking, and so on can be performed in a recursive manner. The process terminates when the masks produce monomodal histograms with the image fully partitioned.

Segmentation techniques based on region growing start with small atomic regions (one or a few pixels) and build coherent pixel regions in a bottom-up fashion.

Local features such as the intensity of a group of pixels relative to the average intensity of neighboring pixels are used as criteria for the merging operation. A low level of contrast between contiguous groups gives rise to the merging of areas, while a higher level of contrast, such as found at boundaries, provides the criteria for region segregation.

Split-and-merge techniques attempt to gain the advantages of both methods. They combine top-down and bottom-up processing using both region splitting and merging until some split-merge criterion no longer exists. At each step in the process, split and merge threshold values can be compared and the appropriate operation performed. In this way, over-splitting and under-merging can be avoided.

14.4 DESCRIBING AND LABELING OBJECTS

We continue in this section with further intermediate-level processing steps all aimed at building higher levels of abstraction. The processing steps here are related to describing and labeling the regions.

Once the image has been segmented into disjointed regions, their shapes, spatial interrelationships, and other characteristics can be described and labeled for subsequent interpretation. This process requires that the outlines or boundaries, vertices, and surfaces of the objects be described in some way. It should be noted, however, that a description for a region can be based on a two- or three-dimensional image interpretation. Initially, we focus on the two-dimensional interpretation.

Typically, a region description will include attributes related to size, shape, and general appearance. For example, some or all of the following features might be included.

- Region area
- Contour length (perimeter) and orientation
- Location of the center of mass
- Minimum bounding rectangle
- Compactness (area divided by perimeter squared)
- Fitted scatter matrix of pixels
- Number and characteristics of holes or internal occlusions
- Minimum bounding rectangle
- Degree and type of texture
- Average intensity (or average intensities of base colors)
- Type of boundary segments (sharp, fuzzy, and so on) and their locations
- Boundary contrast
- Chain code (described below)
- Shape classification number (task specific)
- Position and types of vertices (number of adjoining segments)

Some of the above features are illustrated in Figure 14.16.

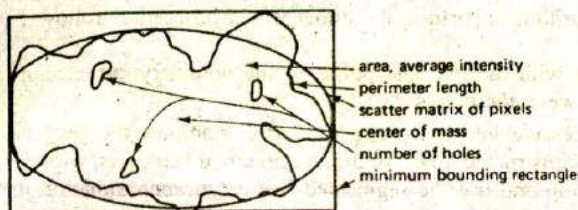


Figure 14.16 Descriptive features for a region.

In addition to these characteristics, the relationships between regions may also be important, particularly adjacent regions. Relations between regions can include their relative positions and orientations, distances between boundaries, intervening regions, relative intensities or contrasts in color, degree of abutment, and degree of connectivity or concentration. When the image domain is known, domain or task specific features may also be useful.

Next, we examine some of the definitions and methods used for region descriptions.

Describing Boundaries

Boundaries can be described as linked straight-line segments, fitted polynomial curves, or in a number of other ways. One simple method of fitting straight-line segments to an arbitrary boundary is by successive linear segmentation fitting. This method permits any degree of fit accuracy at the expense of computation time. The fitting procedure is illustrated in Figure 14.17.

The fitting begins by connecting a single straight line to the two end points and using this as an approximation to the curve (a). Additional lines are then constructed using the points on the curve at maximum perpendicular distances to the fitted lines (b, c, and d).

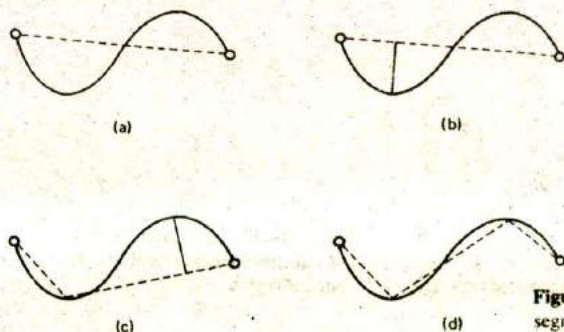


Figure 14.17 Curve fitting with linear segments.

An algorithm to perform the fitting would proceed as follows.

1. Starting with the two end points of the boundary curve, construct a straight line between the points.
2. At successive intervals along the curve, compute the perpendicular distance to the constructed line. If the maximum distance is within some specified limit, stop and use the segmented line as an approximation to the boundary.
3. Otherwise, choose the point on the curve at which the largest distance occurs and use this as a breakpoint with which to construct two new line segments which connect to the two endpoints. Continue the process recursively with each subcurve until the stopping condition of Step 2 is satisfied.

Chain Codes

Another method used for boundary descriptions is known as chain coding. A *chain code* is a sequence of integers which describe the boundary of a region in terms of displacements from some starting point. A chain code is specified by four or more direction numbers which give a trace of the directional displacements of successive unit line segments. An example of a four direction chain code is presented in Figure 14.18.

Chain code descriptions are useful for certain types of object matchings. If the starting position is ignored, the chain code is independent of object location. A derivative or difference ($\text{mod } 4$) for a chain code can also be useful since it is invariant under object rotation. The derivative is found by counting the number of 90 degree counterclockwise rotations made from segment to segment. Thus, the derivative for the chain code of Figure 14.18 is the code

10000303001000103000300003000000000.

Other Descriptive Features

Some other descriptive features include the area, intensity, orientation, center of mass, and bounding rectangle. These descriptions are determined in the following way.

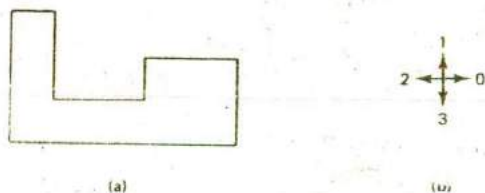


Figure 14.18 (a) Region boundary, (b) direction numbers, and (c) chain code for region boundary.

(c) Chain code: 1111100333000011000033333222222222

1. The area of a region can be given by a count of the number of pixels contained in the region.
2. The average region intensity is just the average gray-level intensity taken over all pixels in the region. If color is used in the image, the average is given as the three base color intensity averages.
3. The center of mass M_c for a region can be computed as the average x - y vector position (denoted as P_i), that is

$$M_c = (1/n) \sum_{i=1}^n P_i$$

4. The scatter matrix S_m is an elliptical area which approximates the shape of the region. It may be computed as the average distance from the center of the region mass as follows.

$$S_m = \sum_{i=1}^n (P_i - M_c) (P_i - M_c)^t$$

where t denotes matrix transposition.

5. The minimum bounding rectangle is found as the rectangular area defined by the intersection of the horizontal and vertical lines which pass through the maximum and minimum pixel positions of the region.

Three-Dimensional Descriptions

Up to this point, we have been mainly concerned with developing two-dimensional descriptions for images. But for many applications, an analysis which produces a three-dimensional scene description will be required. When a stereo system is being used, the methods described in the previous section for stereo analysis can be applied to estimate such parameters as depths, volumes, and distances of objects. When a two-dimensional image is being used as the source, this information must be determined by other means.

Several programs capable of interpreting images consisting of three-dimensional polyhedral blocks world objects were written beginning in the early 1960s (Roberts, 1965; Guzman, 1969; Huffman, 1971; Clowes, 1971 and Waltz, 1975). The experience gained from this work has led to algorithms and techniques with which to classify and identify regular complex polyhedral types of objects from two-dimensional images.

Roberts wrote a program which began by finding lines in the image which corresponded to the edges of the polyhedral objects. It then used descriptions of the lines to match against stored models of primitive objects such as wedges, cubes, and prisms. To perform the match operation, it was necessary to transform the objects by scaling and performing rotations and translations until a best match was

possible. Once a match was obtained and all objects identified, the program demonstrated its "understanding" of the scene by producing a graphic display of it on a monitor screen.

Guzman wrote a program called SEE which examined how surfaces from the same object were linked together. The geometric relationships between different types of line junctions (vertices) helped to determine the object types. Guzman identified eight commonly occurring edge junctions for his three-dimensional blocks world objects. The junctions were used by heuristic rules in his program to classify the different object by type (Figure 14.19).

Huffman and Clowes, working independently, extended this work by developing a line labeling scheme which systematized the classification of polyhedral objects. Their scheme was used to classify edges as either concave, convex, or occluding. Concave edges are produced by two adjacent touching surfaces which produce a concave (less than 180°) depth change. Conversely, convex edges produce a convexly viewed depth change (greater than 180°), and an occluding edge outlines a surface that obstructs other objects.

To label a concave edge, a minus sign is used. Convex edges are labeled with a plus sign, and a right or left arrow is used to label the occluding or boundary edges. By restricting vertices to be the intersection of three object faces (trihedral vertices), it is possible to reduce the number of basic vertex types to only four: the L, the T, the Fork, and the Arrow (Figure 14.20). Different label combinations assigned to these four types then assist in the classification and identification of objects.

When a three-dimensional object is viewed from all possible positions, the four junction types, together with the valid edge labels, give rise to eighteen different permissible junction configurations as depicted in Figure 14.20. From a dictionary of these valid junction types, a program can classify objects by the sequence of bounding vertices which describe it. Impossible object configurations such as the one illustrated in Figure 14.21 can also be detected.

Geometric constraints, together with a consistent labeling scheme, can greatly simplify the object identification process. A set of labeling rules which greatly facilitates this process can be developed for different classes of objects. For example, using the labels described above, the following rules will apply for many polyhedral

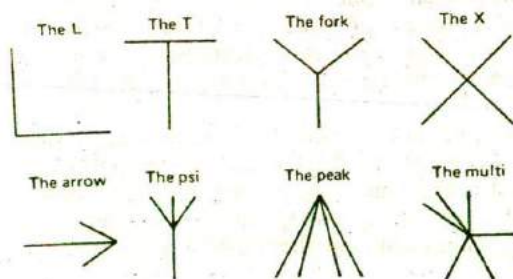


Figure 14.19 Three-dimensional polyhedral junction types.

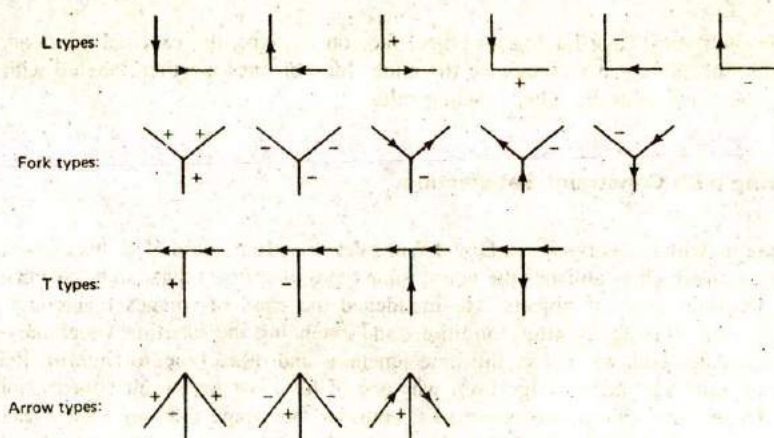


Figure 14.20 Valid junction labels for three-dimensional shapes.

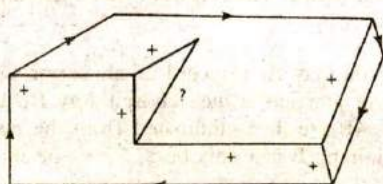


Figure 14.21 Example of an impossible object.

objects: (1) the arrow should be directed to mark boundaries by traversing the object in a clockwise direction (the object face appears on the right of the arrow), (2) unbroken lines should have the same label assigned at both ends, (3) when a fork is labeled with a + edge, it must have all three edges labeled as +, and (4) arrow junctions which have a $\rightarrow$ label on both barb edges must also have a + label on the shaft.

These rules can be applied to a polygonal object as illustrated in Figure 14.22.

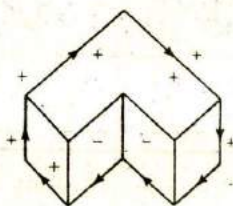


Figure 14.22 Example of object labeling.

Starting with any edge having an object face on its right, the external boundary is labeled with the $\rightarrow$ in a clockwise direction. Interior lines are then labeled with $+$ or $-$ consistent with the other labeling rules.

Filtering with Constraint Satisfaction

Continuing with this early work, David Waltz developed a method of vertex constraint propagation which establishes the permissible types of vertices that can be associated with a certain class of objects. He broadened the class of images that could be analyzed by relaxing lighting conditions and extending the labeling vocabulary to accommodate shadows, some multiline junctions and other types of interior lines.

To see how this procedure works, consider the image drawing of a pyramid as illustrated in Figure 14.23. At the right side of the pyramid are all possible labelings for the four junctions A, B, C, and D.

Using these labels as mutual constraints on connected junctions, permissible labels for the whole pyramid can be determined. The constraint satisfaction procedure works as follows:

1. Starting at an arbitrary junction, say A, a record of all permissible labels is made for that junction. An adjacent junction is then chosen, say B, and labels which are inconsistent with the line AB are then eliminated from the permissible A and B lists. In this case, the line joining B can only be a $+$, $-$, or an up-arrow

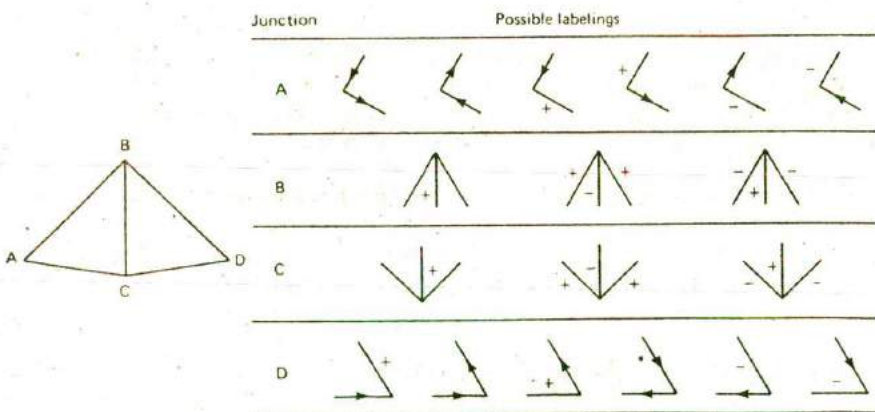


Figure 14.23 Possible labelings for an object.

→. Consequently, two of the possible A labelings can be eliminated with the remaining four being



2. Choosing junction C next, we find that the BC constraints are satisfied by all of the B and C labelings; so no reduction is possible with this step. On the other hand, the line AC must be labeled as $-$ or as an up-left-arrow $\nwarrow$ to be consistent. Therefore, an additional label for A can be eliminated to reduce the remainder to the following.



3. This new restriction on A now permits the elimination of one B labeling to maintain consistency. Thus, the permissible B labelings remaining are now



This reduction in turn, places a new restriction on BC, permitting the elimination of one C label, since BC must now be labeled as a $+$ only. This leaves the remaining C labels as



4. Moving now to junction D, we see that of the six possible D labelings, only three satisfy the BD constraint of a $-$ or a down-arrow. Therefore, the remaining permissible labelings for D are now



Continuing with the above procedure, it will be found that further label eliminations are not possible since all constraints have been satisfied. The above process is completed by finding the different combinations of unique labelings that can be assigned to the figure. This can be accomplished through a tree search process. A simple enumeration of the remaining labels shows that it is possible to find only

three different labelings. We leave the labeling as an exercise at the end of this chapter.

The process of constraint satisfaction described here has been called Waltz filtering. It is a special form of a more general solution process called relaxation in which constraints are iteratively reduced or eliminated until equilibrium is reached. Such processes can be very effective in reducing large search spaces to manageable ones.

Template Matching

Template matching is the process of comparing patterns found in an image with prestored templates that are already named. The matching process may occur at lower levels using individual or groups of pixels using correlation techniques or at higher image processing levels using labeled region structures. Comparisons between object and template may be based on an exact or on a partial match; and, the matching process may use whole or component pieces when comparing the two. Rigid or flexible templates may also be used. (An example of flexible template matching was described in Section 10.5, Partial Matching, where the idea of a rubber mask template was introduced.)

Template matching can be effective only when the search process is constrained in some way. For example, the types of scenes and permissible objects should be known in advance, thereby limiting the possible pattern-template pairs. The use of some form of informed search can help restrict the size of the search space.

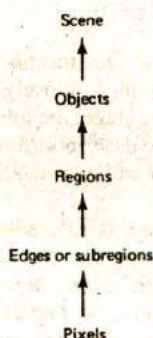
HIGH-LEVEL PROCESSING

Before proceeding with a discussion of the final (high-level) steps in vision processing, we shall briefly review the processing stages up to this point. We began with an image of gray-level or tristimulus color intensity values and digitized this image to obtain an array of numerical pixel values. Next, we used masks or some other transform (such as Fourier) to perform smoothing and edge enhancement operations to reduce the effects of noise and other unwanted features. This was followed by edge detection to outline and segment the image into coherent regions. The product of this step is a primal sketch of the objects. Region splitting and/or merging, the dual of edge finding, can also be used separately or jointly with edge finding as part of the segmentation process.

Histogram computations of intensity values and subsequent analyses were an important part of the segmentation process. They help to establish threshold levels which serve as cues for object separation. Other techniques such as minimum spanning tree or dynamic programming are sometimes used in these early processing stages to aid in edge finding.

Following the segmentation process, regions are analyzed and labeled with their characteristic features. The results of these final steps in intermediate-level

processing is a set of region descriptions (data structures). Such structures are used as the input to the final high-level image processing stage. A summary of the data structures produced from the lowest processing stage to the final interpretation stage then can be depicted as follows.



Marr's Theory of Vision

David Marr and his colleagues (1982, 1980, and 1978) proposed a theory of vision which emphasized the importance of the representational scheme used at each stage of the processing. His proposal was based on the assumption that processing would be carried out in several steps similar to the summary description given above. The steps, and the corresponding representations are summarized as follows.

1. **Gray-level image.** The lowest level in processing consists of the two-dimensional array of pixel intensity levels. These levels correspond to important physical properties of world objects, the illumination, orientation with respect to the observer, geometry, surface reflectances, and object discontinuities. Processing at this stage is local with structure being implicit only. The key aspect of representation at this level is that it facilitate local and first-order statistical transformations.

2. **Raw primal sketch.** The primal sketch is a two-dimensional representation which makes object features more iconic and explicit. It consists of patterns of edge segments, intensity changes, and local features such as texture. The representation at this stage should emphasize pictorial image descriptions and facilitate transformations to the next stage where surface features and volumes are described.

3. **The $2\frac{1}{2}$ -dimensional sketch.** This sketch is an explicit representation of a three-dimensional scene in which objects are given a viewer-centered coordinate system. The models here provide distance, volume, and surface structure. Three-dimensional spatial reconstruction requires the use of various tools including stereopsis, shape contours, shape from shading and texture, and other tools described earlier.

4. The three-dimensional models. The representations at the three-dimensional model level are symbolic ones giving attribute, relational, and geometric descriptions of the scene. The use of generalized cones and cylinders help to represent many object types, and hierarchical descriptions facilitate processing.

High-Level Processing

High-level processing techniques are less mechanical than either of the preceding image processing levels. They are more closely related to classical AI symbolic methods. In the high-level processing stage, the intermediate-level region descriptions are transformed into high-level scene descriptions in one of the knowledge representation formalisms described earlier in Part II (associative nets, frames, FOPL statements, and so on; see Figure 14.24).

The end objective of this stage is to create high-level knowledge structures which can be used by an inference program. Needless to say, the resulting structures should uniquely and accurately describe the important objects in an image including their interrelationships. In this regard, the particular vision application will dictate the appropriate level of detail, and what is considered to be important in a scene description.

There are various approaches to the scene description problem. At one extreme, it will be sufficient to simply apply pattern recognition methods to classify certain objects within a scene. This approach may require no more than application of the methods described in the preceding chapter. At the other extreme, it may be desirable to produce a detailed description of some general scene and provide an interpretation of the function, purpose, intent, and expectations of the objects in the scene. Although this requirement is beyond the current state-of-the-art, we can say that it will require a great many prestored pattern descriptions and much general world knowledge. It will also require improvements on many of the processing techniques described in this chapter.

```
(region6
  (mass-center 23 48)
  (shape-code 24)
  (area 245)
  (number-boundary-segments 6)
  (chain-code 1133300011. . .)
  (orientation 85)
  (borders (region4 (position left-of) (contrast 5))
            (region7 (position above) (contrast 2))

  (mean-intensity 0.6)
  (texture light regular)
```

Figure 14.24 Typical description of a segmented region.

Before a scene can be described in terms of high-level structures, prestored model descriptions of the objects must be available. These descriptions must be compared with the region descriptions created during the intermediate-level stage. The matching process can take the form of rule instantiations, segmented graph or network matchings, frame instantiations, traversal of a discrimination network (decision tree), or even response to message patterns in an object oriented system. The type of matching will naturally be influenced by the representation scheme chosen for the final structures.

To round out this section, we consider some of the approaches used in the high-level processing stage. In the following section, we consider some complete vision system architectures.

Associative networks have become a popular representation scheme for scene descriptions since they show the relationships among the objects as well as object characteristics. A simple example of an outdoor scene representation is illustrated in Figure 14.25.

Scene descriptions such as this can be formulated by interpreting region descriptions of the type shown in Figure 14.16. The interpretation knowledge will be encoded in production rules or other representation scheme. For example, a rule used to identify a sky region in an outdoor color scene would be instantiated with sky properties such as intensity, color, shape, and so forth. A rule to identify houses in an aerial photograph would be instantiated with conditions of area, compactness, texture, type of border, and so on, as illustrated in Figure 14.26. Rule conditions will sometimes have fuzzy or probability predicates to allow for similarity or partial

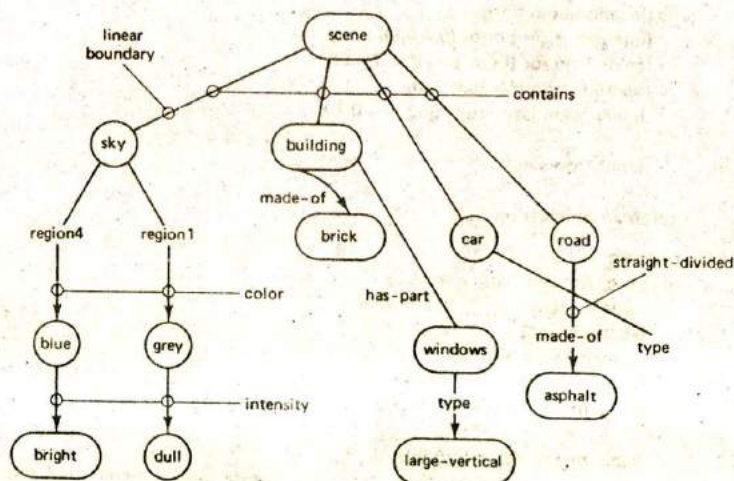


Figure 14.25 Associative network scene representation.

matches rather than absolute ones. Rule conclusions will be rated by likelihood or certainty factors instead of complete certainty. Identification of objects can then be made on the basis of a likelihood score. In Figure 14.26 (a) pairs of numbers are given in the antecedent to suggest acceptable condition levels comparable to Dempster-Shafer probabilities (the values in the figure are arbitrarily chosen with a scale of 0 to 1.0)

When rule-based identification is used, the vision system may be given an initial goal of identifying each region. This can be accomplished with a high-level goal statement of the following type.

```
(label region
  (or (*rgn = building)
      (*rgn = bushes)
      (*rgn = car)
      (*rgn = house)
      (*rgn = road)
      (*rgn = shadow)
      (*rgn = tree)))
```

Other forms of matching may also be used in the interpretation process. For example, a decision tree may be used in which region attributes and relation values determine the branch taken at each node when descending the tree. The leaves of the decision tree are labeled with the object identities as in Figure 14.27.

```
(R10-sky
  (and (location upper *rgn)
        (intensity *rgn bright (0.4 0.8))
        (color *rgn (or (blue grey)) (0.7 1.0))
        (textural *rgn low (0.8 1.0))
        (linear-boundary *rgn rgn2 (0.4 0.7)))
  →
  (label *rgn sky))
```

(a) Sky Identification Rule

```
(R32-building
  (and (intensity-avg *rgn > image)
        (area >= 60)
        (area <= 250)
        (compactness >= 0.6)
        (texture-variation <= 64.0)
        (percent border-planer >= 60))
  →
  (label region HOUSE (0.9)))
```

(b) Building Identification Rule

Figure 14.26 Interpretation rules for a sky and a building.

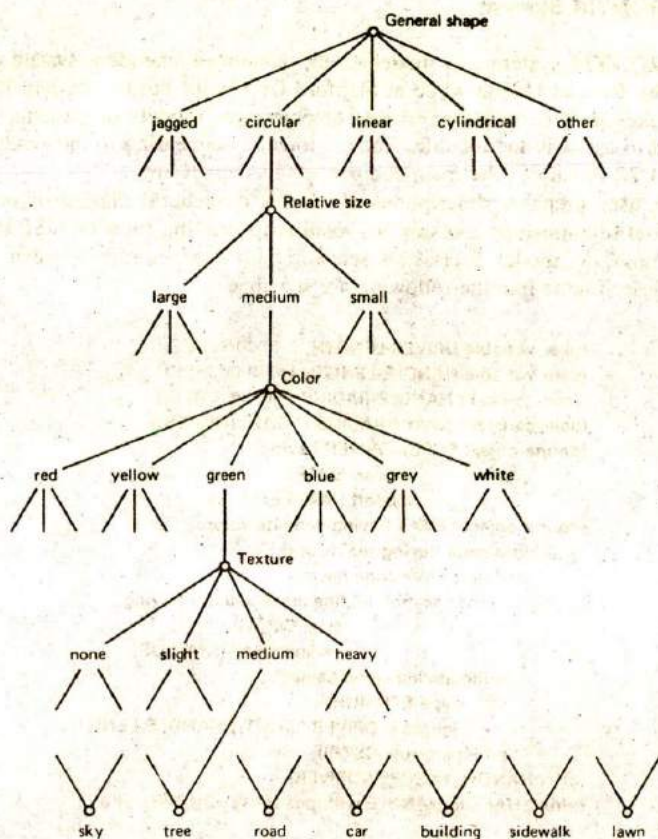


Figure 14.27 Object identification tree.

Objects, with their attributes and relations are then used to construct an associative net scene, a frame network, or other structure.

14.6 VISION SYSTEM ARCHITECTURES

In this section we present two vision systems which are somewhat representative of complete system architectures. The first system is a model-based system, one of the earliest successful vision systems. The second is a color region analyzer recently developed at the University of Kyoto, Japan.

The ACRONYM System

The ACRONYM system is a model-based, domain independent system developed by Rodney Brooks (1981) while at Stanford University during the late 1970s. The system takes user descriptions of sets of objects as models or patterns which are then used to assist in the identification of structures appearing in monocular images. Figure 14.28 illustrates the main components of the system.

The user prepares descriptions of objects or general classes of objects and their spatial relationships and subclass relationships in the form of LISP statements. For example, to model a class of screwdrivers with lengths between 1 and 10 inches, descriptions like the following are specified.

```
(user-variable DRIVER-LENGTH (* 10.0 INCHES))
(user-variable HANDLE-LENGTH (* 4.0 INCHES))
(user-constant HANDLE-RADIUS (* 0.5 INCHES))
(user-constant SHAFT-RADIUS (* 0.125 INCHES))
(define object SCREWDRIVER having
  subpart SHAFT
  subpart HANDLE)
(define object SHAFT having cone-descriptor
  (define cone having main-cone
    (define simple-cone having
      cross-section (define cross-section having
        type CIRCLE
        radius SHAFT-RADIUS)
      spine (define spine having
        type STRAIGHT
        length (- DRIVER-LENGTH HANDLE-LENGTH))
        sweeping-rule CSW)))
(affix HANDLE to SCREWDRIVER)
(affix SHAFT to HANDLE with pos HANDLE-LENGTH 0 0)
```

The user descriptions are parsed and transformed by the system into geometric and algebraic network representations. These representations provide volumetric descriptions in local coordinate systems. A graphic presentation, the system's interpretation of the input models created by the user, provides feedback to the user during the modeling process. The completed representations are used by the system to predict what features (e.g. shape, orientation, and position) of the modeled objects can be observed from the input image components. The predicted models are stored as prediction graphs.

The visual input consists of gray-level image processing arrays, a line finder, and an edge linker. This part of the system provides descriptions of objects as defined by segmented edge structures. The descriptions created from this unit are represented as observation graphs. One output from the predictor serves as an input

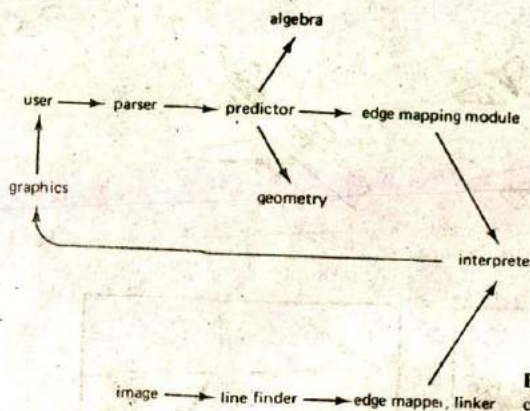


Figure 14.28 Main functional components of ACRONYM.

to the edge mapping and linking module. This unit uses the predicted information (predicted edges, ribbons, or ellipses in the modeled objects) to assist in finding and identifying image objects appearing in the input image. Outputs from both the predictor and the edge mapper and linker serve as inputs to the interpreter. The interpreter is essentially a graph matcher. It tries to find the most matches among subgraphs of the image observation graph and the prediction graph. Each match becomes an interpretation graph. Partial matching is accommodated in the interpretation process through consistency checks.

The basic interpretation process is summarized in Figure 14.29 where models are given for two wide bodied aircraft, (a Boeing 747 and a Lockheed L-1011), and the interpretation of an aircraft from gray-level image to ACRONYM's interpretation is shown.

Ohta's Color Scene Analyzer

Yuichi Ohta of Kyoto University recently developed a vision system which performs region analysis on outdoor color scenes (1986). Outdoor scenes typically include objects such as trees, bushes, sky, roads, buildings, and other objects which are more naturally defined as regions rather than edges. His system makes use of the role color can play in the segmentation process.

Starting with tricolor (red, green, and blue) intensity arrays, digitized images are produced from which regions are defined by a segmentation splitting process. The output of the segmentation process is a two-dimensional array which identifies regions as commonly numbered pixel areas. This array is then transformed into a structured data network which contains descriptive elements for regions such as boundary segments, vertices, and the like. With this network, the system constructs a semantic description of the scene using model knowledge in the form of production rules. Figure 14.30 illustrates the main processing steps carried out by the system.

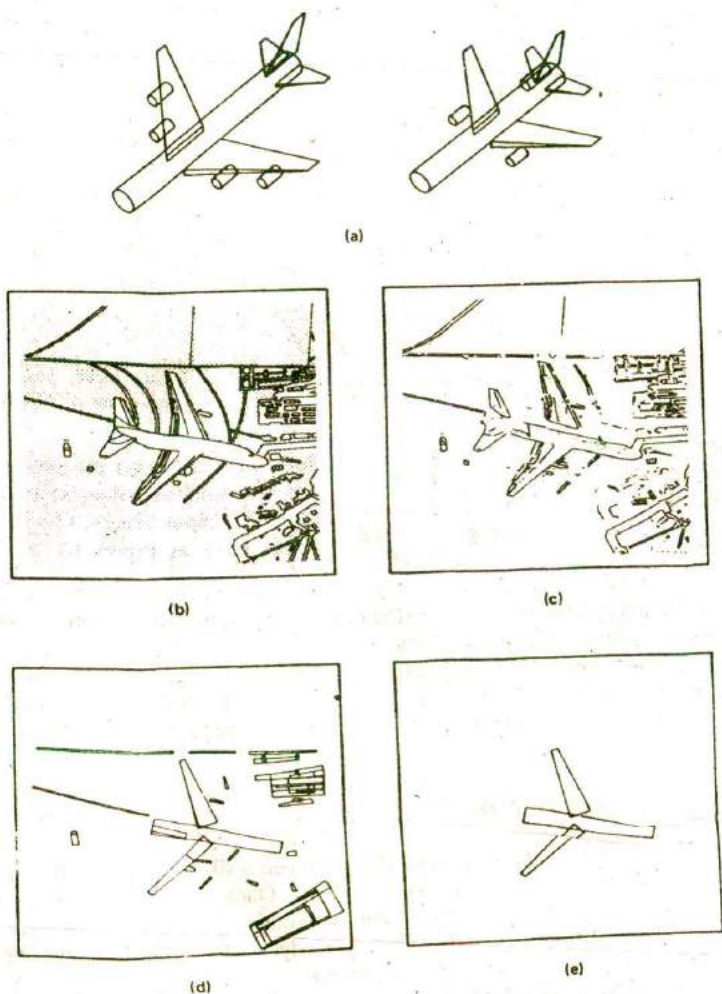


Figure 14.29 Models and stages of interpretation in ACRONYM. (Courtesy of Rodney A. Brooks.)

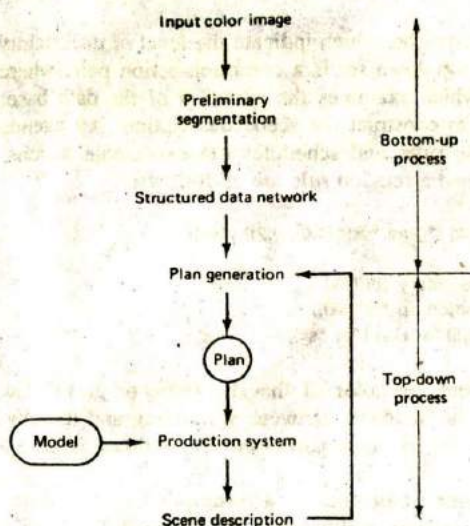


Figure 14.30 Color scene region analyzer.

In the preliminary segmentation stage, the image is segmented into coherent regions using region splitting methods based on color information. Multihistograms serve as cues for thresholding and region splitting. Color features are selected based on results from the Karhunen-Loeve transformation (Devijver and Kittler, 1982) which amounts to choosing those features having the greatest discriminating power (essentially the greatest variance). These segmented regions become the atomic elements from which the structured data network is built.

Regions are characterized by their boundary segments, vertices, line segments, and holes. These basic descriptions are formed during the preliminary segmentation phase. From these, other features are derived including the area, mean color intensities (red, green, blue), degree of texture, contour length, position of mass center, number of holes, minimum bounding rectangle, distance from origin, and orientation. These, and region relationships are described in a data structure known as a patchery data structure. Elements in the data network are essentially matched against model knowledge described in the form of production rules. The rule actions then effectively construct the scene description.

A *plan* is a representation of the crude structure of the input scene given as object labels and their degree of correctness. It is generated by the bottom-up process to provide clues concerning which knowledge can be applied to different parts of the scene.

Knowledge of the task world is represented by sets of production rules. One set is used in the bottom-up process and the other in the top-down process. Each rule in the bottom-up set has a fuzzy predicate which describes properties of relations

between objects. The rules also have weights which indicate the level of uncertainty of the knowledge. Each rule in the top-down set is a condition-action pair, where the condition is a fuzzy predicate which examines the situation of the data base. The action part includes operations to construct the scene description. An agenda manages the activation of production rules and schedules the executable actions. Examples of a typical property rule and a relation rule are as follows:

```
[(GEN (or (*blue *sk) (*gray *sk)) (1.0 . 0.2)) (*sk)]
```

```
[GEN (and (linear-boundary *bl *sk)
  (not (position up *bl *sk))
  (1.0 . 0.5) for sky) (*bl *sk))]
```

The first rule is a property rule about the color of the sky (blue or gray). The second rule is a relation rule about the boundary between a building and the sky. The boundary between the two has a lot of linear parts, and the building is not on the upper side of that boundary.

The final product of the analyzer is, of course, a description of the scene. This is constructed as a hierarchical network as illustrated in Figure 14.31.

Ohta's system has demonstrated that it can deal with fairly complex scenes, including objects with substructures. To validate this claim, a number of outdoor scenes from the Kyoto University campus were analyzed correctly by the system.

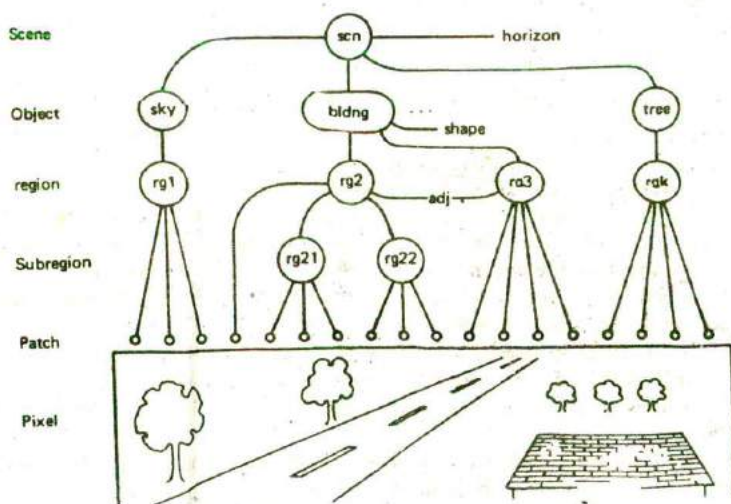


Figure 14.31 Basic structure of the scene description.

14.7 SUMMARY

Computer vision is a computation intensive process. It involves multiple transformations starting with arrays of low level pixels and progressing to high level scene descriptions. The transformation process can be viewed as three stages of processing: low- or early-level processing, intermediate-, and high- or late-level processing. Low-level processing is concerned with the task of finding structure among thousands of atomic gray-level (or tristimulus) pixel intensity values. The goal of this phase is to find and define enough structure in the raw image to permit its segmentation into coherent regions which correspond to definable objects from the source scene. Intermediate-level processing is concerned with the task of accurately forming and describing the segmented regions. The atomic units of this phase are regions and subregions. Finally, high-level processing requires that the segmented regions from the intermediate stage be transformed into scene descriptions. This stage of processing is less mechanical than the two previous stages and relies more on classical AI methods of symbolic processing.

Low-level processing usually involves some form of smoothing operation on the digitized arrays. Smoothing helps to reduce noise and other unwanted features. This is followed by some form of edge detection such as the application of difference operators to the arrays. Edge fragments must then be joined to form continuous contours which outline objects. Various techniques are available for these operations.

The dual-of-the-edge-finding approach is region segmentation, which may be accomplished by region splitting, region growing, or a combination of both. Multithresholding and thresholding are commonly used techniques in the segmentation process. They are applied to one or more image features such as intensity, color, texture, shading, or optical flow in the definition of coherent regions. The end product of the segmentation process will be homogeneous regions. The properties of these regions and their interrelationships must be described in order that they may be identified in the high level processing stage. Regions can be described by boundary segments, vertices, number of holes, compactness, location, orientation, and so on.

The final stage is the knowledge application stage, since the regions must be interpreted and explained. This requires task or domain specific knowledge as well as some general world knowledge. The current state-of-the-art in computer vision does not permit the interpretation of arbitrary complex scenes such as that depicted in Figure 14.1. Much work still remains before that degree of sophistication can be realized.

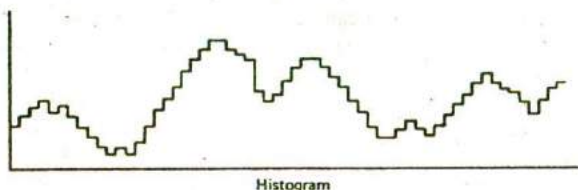
EXERCISES

- 14.1. Visual continuity in TV systems is maintained through the generation of 15 frames per second. What characteristics must an ADC unit have to match this same level of continuity for a vision system with a 1024×1024 pixel resolution?

- 14.2. Describe the types of world knowledge a vision system must have to "comprehend" the scene portrayed in Figure 14.3.
- 14.3. Suppose the CPU in a vision system takes 200 nanoseconds to perform memory/register transfers and 500 nanoseconds to perform basic arithmetic operations. Estimate the time required to produce a binary image for a system with a resolution of 256×256 pixels.
- 14.4. How much memory is required to produce and compare five different binary images, each with a different threshold level? Assume a system resolution of 512×512 . Can the binary images be compressed in some way to reduce memory requirements?
- 14.5. Find the binary image for the array given below when the threshold is set at 35.

23	1	32	35
36	30	42	38
2	9	34	36
37	36	35	33

- 14.6. Given the following histogram, what are the most likely threshold points? Explain why you chose the given points and rejected others.



- 14.7. What is the value of the smoothed pixel for the associated mask?

MASK				PIXELS		
		3/16		7	8	9
3/16	1/4		3/16	5	4	6
		3/16		4	6	2

- 14.8. Compare the effects of the eight- and four-neighbor filters described in Section 14.2 when applied to the following array of pixel gray-level values.

5	8	8	10	12	29	32	30
4	7	8	9	10	9	30	29
5	8	7	8	11	33	31	34
6	9	8	10	34	31	29	33
6	8	9	32	30	29	5	6
8	7	31	32	32	28	6	7
7	8	33	33	29	7	8	7
9	30	32	31	28	8	8	9

- 14.9. Low noise systems should use little or no filtering to avoid unnecessary blurring. This means that more weight should be given to the pixel being smoothed. Define two low-noise filters, one a four-neighbor and one an eight-neighbor filter, and compare their effects on the array of Problem 14.5.
- 14.10. Using a value of $n = 1$, apply D_x and D_y (horizontally) to the array of Problem 14.5 and comment on the trace of any apparent edges.
- 14.11. Apply the vector gradient to the array of Problem 14.5 and compare the results to those of Problem 14.7.
- 14.12. This problem relates to the application of template matching using correlation techniques. The objective is to try to match an unknown two-dimensional curve or waveform with a known waveform. Assume that both waveforms are discrete and are represented as arrays of unsigned numbers. Write a program in any suitable language to match the unknown waveform to the known waveform using the correlation function given as

$$C_i = \frac{\langle \mathbf{X}, \mathbf{Z}_i \rangle}{\|\mathbf{X}\| \|\mathbf{Z}_i\|}$$

where $\mathbf{X}$ is the unknown pattern vector, $\mathbf{Z}_i$ is the known pattern vector at position i , $\langle \mathbf{X}, \mathbf{Z}_i \rangle$ denotes the inner product of $\mathbf{X}$ and $\mathbf{Z}_i$, and $\|\mathbf{X}\|$ is the norm of $\mathbf{X}$.

$$\|\mathbf{X}\| = [\sum x_i^2]^{1/2}$$

- 14.13 Write a program to apply the Sobel edge detection mask to an array consisting of 256×256 pixel gray level values.
- 14.14 Color and texture are both potentially useful in defining regions. Describe an algorithm that could be used to determine regions that are homogenous in color.
- 14.15 Referring to Problem 14.14, develop an algorithm that can be used to define regions that are homogeneous in texture.
- 14.16 Referring to the two previous problems, develop an algorithm that determines regions on the basis of homogeneity in both color and texture.

Expert Systems Architectures

This chapter describes the basic architectures of knowledge-based systems with emphasis placed on expert systems. Expert systems are a recent product of artificial intelligence. They began to emerge as university research systems during the early 1970s. They have now become one of the more important innovations of AI since they have been shown to be successful commercial products as well as interesting research tools.

Expert systems have proven to be effective in a number of problem domains which normally require the kind of intelligence possessed by a human expert. The areas of application are almost endless. Wherever human expertise is needed to solve problems, expert systems are likely candidates for application. Application domains include law, chemistry, biology, engineering, manufacturing, aerospace, military operations, finance, banking, meteorology, geology, geophysics, and more. The list goes on and on.

In this chapter we explore expert system architectures and related building tools. We also look at a few of the more important application areas as well. The material is intended to acquaint the reader with the basic concepts underlying expert systems and to provide enough of the fundamentals needed to build basic systems or pursue further studies and conduct research in the area.

15.1 INTRODUCTION

An expert system is a set of programs that manipulate encoded knowledge to solve problems in a specialized domain that normally requires human expertise. An expert system's knowledge is obtained from expert sources and coded in a form suitable for the system to use in its inference or reasoning processes. The expert knowledge must be obtained from specialists or other sources of expertise, such as texts, journal articles, and data bases. This type of knowledge usually requires much training and experience in some specialized field such as medicine, geology, system configuration, or engineering design. Once a sufficient body of expert knowledge has been acquired, it must be encoded in some form, loaded into a knowledge base, then tested, and refined continually throughout the life of the system.

Characteristic Features of Expert Systems

Expert systems differ from conventional computer systems in several important ways:

1. Expert systems use knowledge rather than data to control the solution process. "In the knowledge lies the power" is a theme repeatedly followed and supported throughout this book. Much of the knowledge used is heuristic in nature rather than algorithmic.

2. The knowledge is encoded and maintained as an entity separate from the control program. As such, it is not compiled together with the control program itself. This permits the incremental addition and modification (refinement) of the knowledge base without recompilation of the control programs. Furthermore, it is possible in some cases to use different knowledge bases with the same control programs to produce different types of expert systems. Such systems are known as expert system shells since they may be loaded with different knowledge bases.

3. Expert systems are capable of explaining how a particular conclusion was reached, and why requested information is needed during a consultation. This is important as it gives the user a chance to assess and understand the system's reasoning ability, thereby improving the user's confidence in the system.

4. Expert systems use symbolic representations for knowledge (rules, networks, or frames) and perform their inference through symbolic computations that closely resemble manipulations of natural language. (An exception to this is the expert system based on neural network architectures.)

5. Expert systems often reason with metaknowledge; that is, they reason with knowledge about themselves, and their own knowledge limits and capabilities.

Background History

Expert systems first emerged from the research laboratories of a few leading U.S. universities during the 1960s and 1970s. They were developed as specialized problem

solvers which emphasized the use of knowledge rather than algorithms and general search methods. This approach marked a significant departure from conventional AI systems architectures at the time. The accepted direction of researchers then was to use AI systems that employed general problem solving techniques such as hill-climbing or means-end analysis (Chapter 9) rather than specialized domain knowledge and heuristics. This departure from the norm proved to be a wise choice. It led to the development of a new class of successful systems and special system designs.

The first expert system to be completed was DENDRAL, developed at Stanford University in the late 1960s. This system was capable of determining the structure of chemical compounds given a specification of the compound's constituent elements and mass spectrometry data obtained from samples of the compound. DENDRAL used heuristic knowledge obtained from experienced chemists to help constrain the problem and thereby reduce the search space. During tests, DENDRAL discovered a number of structures previously unknown to expert chemists.

As researchers gained more experience with DENDRAL, they found how difficult it was to elicit expert knowledge from experts. This led to the development of Meta-DENDRAL, a learning component for DENDRAL which was able to learn rules from positive examples, a form of inductive learning described later in detail (Chapters 18 and 19).

Shortly after DENDRAL was completed, the development of MYCIN began at Stanford University. MYCIN is an expert system which diagnoses infectious blood diseases and determines a recommended list of therapies for the patient. As part of the Heuristic Programming Project at Stanford, several projects directly related to MYCIN were also completed including a knowledge acquisition component called THEIRESIUS, a tutorial component called GUIDON, and a shell component called EMYCIN (for Essential MYCIN). EMYCIN was used to build other diagnostic systems including PUFF, a diagnostic expert for pulmonary diseases. EMYCIN also became the design model for several commercial expert system building tools.

MYCIN's performance improved significantly over a several year period as additional knowledge was added. Tests indicate that MYCIN's performance now equals or exceeds that of experienced physicians. The initial MYCIN knowledge base contained about only 200 rules. This number was gradually increased to more than 600 rules by the early 1980s. The added rules significantly improved MYCIN's performance leading to a 65% success record which compared favorably with experienced physicians who demonstrated only an average 60% success rate (Lenat, 1984). (An example of MYCIN's rules is given in Section 4.9, and the treatment of uncertain knowledge by MYCIN is described in Section 6.5.)

Other early expert system projects included PROSPECTOR, a system that assists geologists in the discovery of mineral deposits, and R1 (aka XCON), a system used by the Digital Equipment Corporation to select and configure components of complex computer systems. Since the introduction of these early expert systems, numerous commercial and military versions have been completed with a high degree of success. Some of these application areas are itemized below.

Applications

Since the introduction of these early expert systems, the range and depth of applications has broadened dramatically. Applications can now be found in almost all areas of business and government. They include such areas as

- Different types of medical diagnoses (internal medicine, pulmonary diseases, infectious blood diseases, and so on)
- Diagnosis of complex electronic and electromechanical systems
- Diagnosis of diesel electric locomotion systems
- Diagnosis of software development projects
- Planning experiments in biology, chemistry, and molecular genetics
- Forecasting crop damage
- Identification of chemical compound structures and chemical compounds
- Location of faults in computer and communications systems
- Scheduling of customer orders, job shop production operations, computer resources for operating systems, and various manufacturing tasks
- Evaluation of loan applicants for lending institutions
- Assessment of geologic structures from dip meter logs
- Analysis of structural systems for design or as a result of earthquake damage
- The optimal configuration of components to meet given specifications for a complex system (like computers or manufacturing facilities)
- Estate planning for minimal taxation and other specified goals
- Stock and bond portfolio selection and management
- The design of very large scale integration (VLSI) systems
- Numerous military applications ranging from battlefield assessment to ocean surveillance
- Numerous applications related to space planning and exploration
- Numerous areas of law including civil case evaluation, product liability, assault and battery, and general assistance in locating different law precedents
- Planning curricula for students
- Teaching students specialized tasks (like trouble shooting equipment faults)

Importance of Expert Systems

The value of expert systems was well established by the early 1980s. A number of successful applications had been completed by then and they proved to be cost effective. An example which illustrates this point well is the diagnostic system developed by the Campbell Soup Company.

Campbell Soup uses large sterilizers or cookers to cook soups and other canned

products at eight plants located throughout the country. Some of the larger cookers hold up to 68,000 cans of food for short periods of cooking time. When difficult maintenance problems occur with the cookers, the fault must be found and corrected quickly or the batch of foods being prepared will spoil. Until recently, the company had been depending on a single expert to diagnose and cure the more difficult problems, flying him to the site when necessary. Since this individual will retire in a few years taking his expertise with him, the company decided to develop an expert system to diagnose these difficult problems.

After some months of development with assistance from Texas Instruments, the company developed an expert system which ran on a PC. The system has about 150 rules in its knowledge base with which to diagnose the more complex cooker problems. The system has also been used to provide training to new maintenance personnel. Cloning multiple copies for each of the eight locations cost the company only a few pennies per copy. Furthermore, the system cannot retire, and its performance can continue to be improved with the addition of more rules. It has already proven to be a real asset to the company. Similar cases now abound in many diverse organizations.

15.2 RULE-BASED SYSTEM ARCHITECTURES

The most common form of architecture used in expert and other types of knowledge-based systems is the production system, also called the rule-based system. This type of system uses knowledge encoded in the form of production rules, that is, if . . . then rules. We may remember from Chapter 4 that rules have an antecedent or condition part, the left-hand side, and a conclusion or action part, the right-hand side.

IF: Condition-1 and Condition-2 and Condition-3
THEN: Take Action-4

IF: The temperature is greater than 200 degrees, and
The water level is low
THEN: Open the safety valve.

A & B & C & D → E & F

Each rule represents a small chunk of knowledge relating to the given domain of expertise. A number of related rules collectively may correspond to a chain of inferences which lead from some initially known facts to some useful conclusions. When the known facts support the conditions in the rule's left side, the conclusion or action part of the rule is then accepted as known (or at least known with some degree of certainty). Examples of some typical expert system rules were described in earlier sections (for example see Sections 4.9, 6.5, and 10.6).

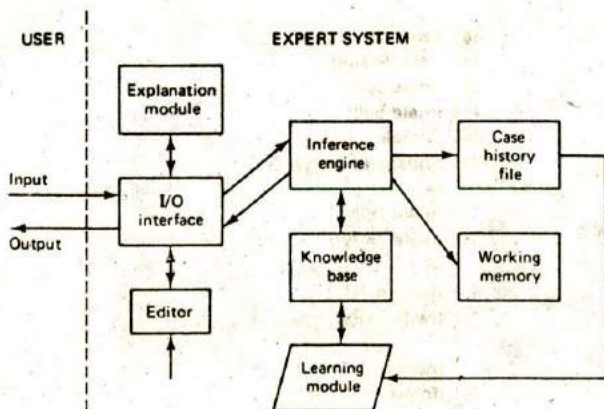


Figure 15.1 Components of a typical expert system.

Inference in production systems is accomplished by a process of chaining through the rules recursively, either in a forward or backward direction, until a conclusion is reached or until failure occurs. The selection of rules used in the chaining process is determined by matching current facts against the domain knowledge or variables in rules and choosing among a candidate set of rules the ones that meet some given criteria, such as specificity. The inference process is typically carried out in an interactive mode with the user providing input parameters needed to complete the rule chaining process.

The main components of a typical expert system are depicted in Figure 15.1. The solid lined boxes in the figure represent components found in most systems whereas the broken lined boxes are found in only a few such systems.

The Knowledge Base

The knowledge base contains facts and rules about some specialized knowledge domain. An example of a simple knowledge base giving family relationships is illustrated in Figure 15.2. The rules in this figure are given in the same LISP format as those of Section 10.6 which is similar to the format given in the OPS5 language as presented by Bronston, Farrell, Kant, and Martin (1985). Each fact and rule is identified with a name ($a1, a2, \dots, r1, r2, \dots$). For ease in reading, the left side is separated from the right by the implication symbol $\rightarrow$. Conjuncts on the left are given within single parentheses (sublists), and one or more conclusions may follow the implication symbol. Variables are identified as a symbol preceded by a question mark. It should be noted that rules found in real working systems may have many conjuncts in the LHS. For example, as many as eight or more are not uncommon.


```

(a1 (male bob))
(a2 (female sue))
(a3 (male sam))
(a4 (male bill))
(a5 (female pam))
(r1 ((husband ?x ?y))
  →
  (male ?x))
(r2 ((wife ?x ?y))
  →
  (female ?x))
(r3 ((wife ?x ?y))
  →
  (husband ?y ?x))
(r4 ((mother ?x ?y)
  (husband ?z ?x))
  →
  (father ?z ?y))
(r5 ((father ?x ?y)
  (wife ?z ?x))
  →
  (mother ?z ?y))
(r6 ((husband ?x ?y))
  →
  (wife ?y ?x))
(r7 ((father ?x ?z)
  (mother ?y ?z))
  →
  (husband ?x ?y))
(r8 ((father ?x ?z)
  (mother ?y ?z))
  →
  (wife ?y ?z))
(r9 ((father ?x ?y)
  (father ?y ?z))
  →
  (grandfather ?x ?z))

```

Figure 15.2 Facts and rules in a simple knowledge base.

In PROLOG, rules are written naturally as clauses with both a head and body. For example, a rule about a patient's symptoms and the corresponding diagnosis of hepatitis might read in English as the rule

IF: The patient has a chronic disorder, and
 the sex of the patient is female, and
 the age of the patient is less than 30, and
 the patient shows condition A, and
 test B reveals biochemistry condition C

THEN: conclude the patient's diagnosis is autoimmune-chronic-hepatitis.

This rule could be written straightaway in PROLOG as

```
conclude(patient, diagnosis, autoimmune.chronic.hepatitis):-  
    same(patient, disorder, chronic),  
    same(patient, sex, female),  
    lessthan(patient, age, 30),  
    same(patient, symptom.a, value.a),  
    same(patient, biochemistry, value.c).
```

Note that PROLOG rules have at most one conclusion clause.

The Inference Process

The inference engine accepts user input queries and responses to questions through the I/O interface and uses this dynamic information together with the static knowledge (the rules and facts) stored in the knowledge base. The knowledge in the knowledge base is used to derive conclusions about the current case or situation as presented by the user's input.

The inferring process is carried out recursively in three stages: (1) match, (2) select, and (3) execute. During the match stage, the contents of working memory are compared to facts and rules contained in the knowledge base. When consistent matches are found, the corresponding rules are placed in a conflict set. To find an appropriate and consistent match, substitutions (instantiations) may be required. Once all the matched rules have been added to the conflict set during a given cycle, one of the rules is selected for execution. The criteria for selection may be most recent use, rule condition specificity, (the number of conjuncts on the left), or simply the smallest rule number. The selected rule is then executed and the right-hand side or action part of the rule is then carried out. Figure 15.3 illustrates this match-select-execute cycle.

As an example, suppose the working memory contains the two clauses

```
(father bob sam)  
(mother sue sam)
```

When the match part of the cycle is attempted, a consistent match will be made between these two clauses and rules r7 and r8 in the knowledge base. The match is made by substituting Bob for ?x, Sam for ?z, and Sue for ?y. Consequently, since all the conditions on the left of both r7 and r8 are satisfied, these two rules will be placed in the conflict set. If there are no other working memory clauses to match, the selection step is executed next. Suppose, for one or more of the selection criteria stated above, r7 is the rule chosen to execute. The clause on the right side of r7 is instantiated and the execution step is initiated. The execution step may result in the right-hand clause (husband bob sue) being placed in working memory or it may be used to trigger a message to the user. Following the execution step, the match-select-execute cycle is repeated.

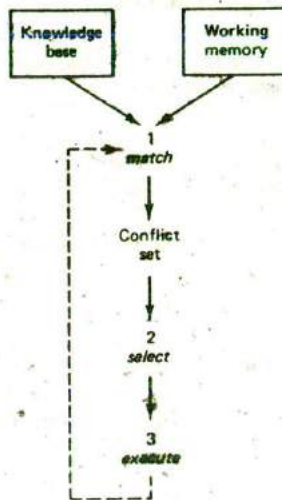


Figure 15.3 The production system inference cycle.

As another example of matching, suppose the two facts (a6 (father sam bill)) and (a7 (father bill pam)) have been added to the knowledge base and the immediate goal is a query about Pam's grandfather. When made, assume this query has resulted in placement of the clause (grandfather ?x pam) into working memory. For this goal to succeed, consistent substitutions must be made for the variables ?x and ?y in rule r9 with a6 and a7. This will be the case if Sam and Bill are substituted for ?x and ?y in the subgoal left-hand conditions of r9. The right hand side will then correctly state that Pam's grandfather is Sam.

When the left side of a sequence of rules is instantiated first and the rules are executed from left to right, the process is called forward chaining. This is also known as data-driven inference since input data are used to guide the direction of the inference process. For example, we can chain forward to show that when a student is encouraged, is healthy, and has goals, the student will succeed.

```

ENCOURAGED(student) → MOTIVATED(student)
MOTIVATED(student) & HEALTHY(student) → WORKHARD(student)
WORKHARD(student) & HASGOALS(student) → EXCELL(student)
EXCELL(student) → SUCCEED(student)
  
```

On the other hand, when the right side of the rules is instantiated first, the left-hand conditions become subgoals. These subgoals may in turn cause sub-subgoals to be established, and so on until facts are found to match the lowest subgoal conditions. When this form of inference takes place, we say that backward chaining is performed. This form of inference is also known as goal-driven inference since an initial goal establishes the backward direction of the inferring.

For example, in MYCIN the initial goal in a consultation is "Does the patient have a certain disease?" This causes subgoals to be established such as "are certain bacteria present in the patient?" Determining if certain bacteria are present may require such things as tests on cultures taken from the patient. This process of setting up subgoals to confirm a goal continues until all the subgoals are eventually satisfied or fail. If satisfied, the backward chain is established thereby confirming the main goal.

When rules are executed, the resulting action may be the placement of some new facts in working memory, a request for additional information from the user, or simply the stopping of the search process. If the appropriate knowledge has been stored in the knowledge base and all required parameter values have been provided by the user, conclusions will be found and will be reported to the user. The chaining continues as long as new matches can be found between clauses in the working memory and rules in the knowledge base. The process stops when no new rules can be placed in the conflict set.

Some systems use both forward and backward chaining, depending on the type of problem and the information available. Likewise, rules may be tested exhaustively or selectively, depending on the control structure. In MYCIN, rules in the KB are tested exhaustively. However, when the number of rules exceeds a few hundred, this can result in an intolerable amount of searching and matching. In such cases, techniques such as those found in the RETE algorithm (Chapter 10) may be used to limit the search.

Many expert systems must deal with uncertain information. This will be the case when the evidence supporting a conclusion is vague, incomplete, or otherwise uncertain. To accommodate uncertainties, some form of probabilities, certainty factors, fuzzy logic, heuristics, or other methods must be introduced into the inference process. These methods were introduced in Chapters 5 and 6. The reader is urged at this time to review those methods to see how they may be applied to expert systems.

Explaining How or Why

The explanation module provides the user with an explanation of the reasoning process when requested. This is done in response to a how query or a why query.

To respond to a how query, the explanation module traces the chain of rules fired during a consultation with the user. The sequence of rules that led to the conclusion is then printed for the user in an easy to understand human-language style. This permits the user to actually see the reasoning process followed by the system in arriving at the conclusion. If the user does not agree with the reasoning steps presented, they may be changed using the editor.

To respond to a why query, the explanation module must be able to explain why certain information is needed by the inference engine to complete a step in the reasoning process before it can proceed. For example, in diagnosing a car that will not start, a system might be asked why it needs to know the status of the

distributor spark. In response, the system would reply that it needs this information to determine if the problem can be isolated to the ignition system. Again, this information allows the user to determine if the system's reasoning steps appear to be sound. The explanation module programs give the user the important ability to follow the inferencing steps at any time during the consultation.

Building a Knowledge Base

The editor is used by developers to create new rules for addition to the knowledge base, to delete outmoded rules, or to modify existing rules in some way. Some of the more sophisticated expert system editors provide the user with features not found in typical text editors, such as the ability to perform some types of consistency tests for newly created rules, to add missing conditions to a rule, or to reformat a newly created rule. Such systems also prompt the user for missing information, and provide other general guidance in the KB creation process.

One of the most difficult tasks in creating and maintaining production systems is the building and maintaining of a consistent but complete set of rules. This should be done without adding redundant or unnecessary rules. Building a knowledge base requires careful planning, accounting, and organization of the knowledge structures. It also requires thorough validation and verification of the completed knowledge base, operations which have yet to be perfected. An "intelligent" editor can greatly simplify the process of building a knowledge base.

TEIRESIAS (Davis, 1982) is an example of an intelligent editor developed to assist users in building a knowledge base directly without the need for an intermediary knowledge engineer. TEIRESIAS was developed to work with systems like MYCIN in providing a direct user-to-system dialog. TEIRESIAS assists the user in formulating, checking, and modifying rules for inclusion in the performance program's knowledge base. For this, TEIRESIAS uses some metaknowledge, that is, knowledge about MYCIN's knowledge. The dialog is carried out in a near English form so that the user needs to know little about the internal form of the rules.

The I/O Interface

The input-output interface permits the user to communicate with the system in a more natural way by permitting the use of simple selection menus or the use of a restricted language which is close to a natural language. This means that the system must have special prompts or a specialized vocabulary which encompasses the terminology of the given domain of expertise. For example, MYCIN can recognize many medical terms in addition to various common words needed to communicate. For this, MYCIN has a vocabulary of some 2000 words.

Personal Consultant Plus, a commercial PC version of the MYCIN architecture, uses menus and English prompts to communicate with the user. The prompts, written in standard English, are provided by the developer during the system building stage. How and why explanations are also given in natural language form.

The learning module and history file are not common components of expert systems. When they are provided, they are used to assist in building and refining the knowledge base. Since learning is treated in great detail in later chapters, no description is given here.

15.3 NONPRODUCTION SYSTEM ARCHITECTURES

Other, less common expert system architectures (although no less important) are those based on nonproduction rule-representation schemes. Instead of rules, these systems employ more structured representation schemes like associative or semantic networks, frame and rule structures, decision trees, or even specialized networks like neural networks. In this section we examine some typical system architectures based on these methods.

Associative or Semantic Network Architectures

Associative or semantic network representation schemes were discussed in Chapter 7. From the description there, we know that an associative network is a network made up of nodes connected by directed arcs. The nodes represent objects, attributes, concepts, or other basic entities, and the arcs, which are labeled, describe the relationship between the two nodes they connect. Special network links include the ISA and HASPART links which designate an object as being a certain type of object (belonging to a class of objects) and as being a subpart of another object, respectively.

Associative network representations are especially useful in depicting hierarchical knowledge structures, where property inheritance is common. Objects belonging to a class of other objects may inherit many of the characteristics of the class. Inheritance can also be treated as a form of default reasoning. This facilitates the storage of information when shared by many objects as well as the inferencing process.

Associative network representations are not a popular form of representation for standard expert systems. More often, these network representations are used in natural language or computer vision systems or in conjunction with some other form of representation.

One expert system based on the use of an associative network representation is CASNET (Causal Associational Network), which was developed at Rutgers University during the early 1970s (Weiss et al., 1978). CASNET is used to diagnose and recommend treatment for glaucoma, one of the leading causes of blindness.

The network in CASNET is divided into three planes or types of knowledge as depicted in Figure 15.4. The different knowledge types are

- Patient observations (tests, symptoms, other signs)
- Pathophysiological states
- Disease categories

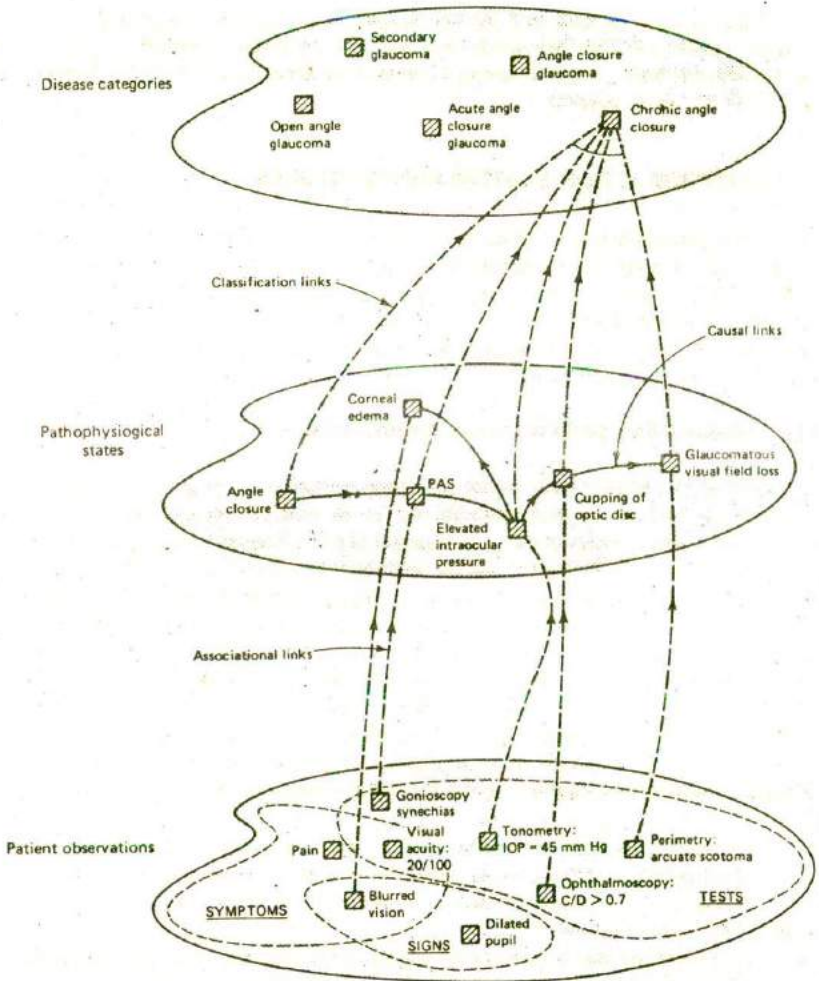


Figure 15.4 Levels of network description in CASNET. (From Artificial Intelligence Journal, Vol. II p. 148, 1978. By permission.)

Patient observations are provided by the user during an interactive session with the system. The system presents menu type queries, and the user selects one of several possible choices. These observations help to establish an abnormal condition caused by a disease process. The condition is established through the causal network

model as part of the cause and effect relationship relating symptoms and other signs to diseases.

Inference is accomplished by traversing the network, following the most plausible paths of causes and effects. Once a sufficiently strong path has been determined through the network, diagnostic conclusions are inferred using classification tables that interpret patterns of the causal network. These tables are similar to rule interpretations.

The CASNET system was never used much beyond the initial research stage. At the time, physicians were reluctant to use computer systems in spite of performance tests in which CASNET scored well.

Frame Architectures

Frame representations were described in Chapter 7. Frames are structured sets of closely related knowledge, such as an object or concept name, the object's main attributes and their corresponding values, and possibly some attached procedures (if-needed, if-added, if-removed procedures). The attributes, values, and procedures are stored in specified slots and slot facets of the frame. Individual frames are usually linked together as a network much like the nodes in an associative network. Thus, frames may have many of the features of associative networks, namely, property inheritance and default reasoning. Several expert systems have been constructed with frame architectures, and a number of building tools which create and manipulate frame structured systems have been developed.

An example of a frame-based system is the PIP system (Present Illness Program) developed at M.I.T. during the late 1970s and 1980s (Szolovits and Pauker, 1978). This system was used to diagnose patients using low cost, easily obtained information, the type of information obtained by a general practitioner during an office examination.

The medical knowledge in PIP is organized in frame structures, where each frame is composed of categories of slots with names such as

- Typical findings
- Logical decision criteria
- Complimentary relations to other frames
- Differential diagnosis
- Scoring

The patient findings are matched against frames, and when a close match is found, a trigger status occurs. A trigger is a finding that is so strongly related to a disorder that the system regards it as an active hypothesis, one to be pursued further. A special is-sufficient slot is used to confirm the presence of a disease when key findings correlate with the slot contents.

Decision Tree Architectures

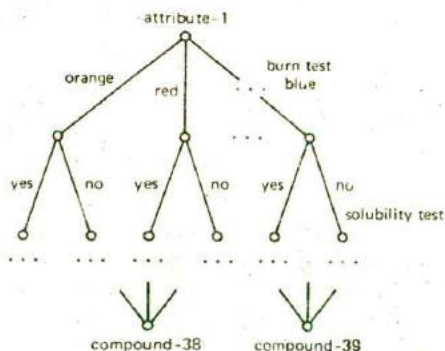
Knowledge for expert systems may be stored in the form of a decision tree when the knowledge can be structured in a top-to-bottom manner. For example, the identification of objects (equipment faults, physical objects, diseases, and the like) can be made through a decision tree structure. Initial and intermediate nodes in the tree correspond to object attributes, and terminal nodes correspond to the identities of objects. Attribute values for an object determine a path to a leaf node in the tree which contains the object's identification. Each object attribute corresponds to a nonterminal node in the tree and each branch of the decision tree corresponds to an attribute value or set of values.

A segment of a decision tree knowledge structure taken from an expert system used to identify objects such as liquid chemical waste products is illustrated in Figure 15.5 (Patterson, 1987). Each node in the tree corresponds to an identifying attribute such as molecular weight, boiling point, burn test color, or solubility test results. Each branch emanating from a node corresponds to a value or range of values for the attribute such as 20–37 degrees C, yellow, or nonsoluble in sulphuric acid.

An identification is made by traversing a path through the tree (or network) until the path leads to a unique leaf node which corresponds to the unknown object's identity.

The knowledge base, which is the decision tree for an identification system, can be constructed with a special tree-building editor or with a learning module. In either case, a set of the most discriminating attributes for the class of objects being identified should be selected. Only those attributes that discriminate well among different objects need be used. Permissible values for each of the attributes are grouped into separable sets, and each such set determines a branch from the attribute node to the next node.

New nodes and branches can be added to the tree when additional attributes



15.5 A segment of a decision tree structure.

are needed to further discriminate among new objects. As the system gains experience, the values associated with the branches can be modified for more accurate results.

Blackboard System Architectures

Blackboard architectures refer to a special type of knowledge-based system which uses a form of opportunistic reasoning. This differs from pure forward or pure backward chaining in production systems in that either direction may be chosen dynamically at each stage in the problem solution process. Other reasoning methods (model driven, for example) may also be used.

Blackboard systems are composed of three functional components as depicted in Figure 15.6.

1. There are a number of *knowledge sources* which are separate and independent sets of coded knowledge. Each knowledge source may be thought of as a specialist in some limited area needed to solve a given subset of problems. The sources may contain knowledge in the form of procedures, rules, or other schemes.
2. A globally accessible data base structure, called a *blackboard*, contains the current problem state and information needed by the knowledge sources (input data, partial solutions, control data, alternatives, final solutions). The knowledge sources make changes to the blackboard data that incrementally lead to a solution. Communication and interaction between the knowledge sources takes place solely through the blackboard.
3. *Control information* may be contained within the sources, on the blackboard, or possibly in a separate module. (There is no actual control unit specified as

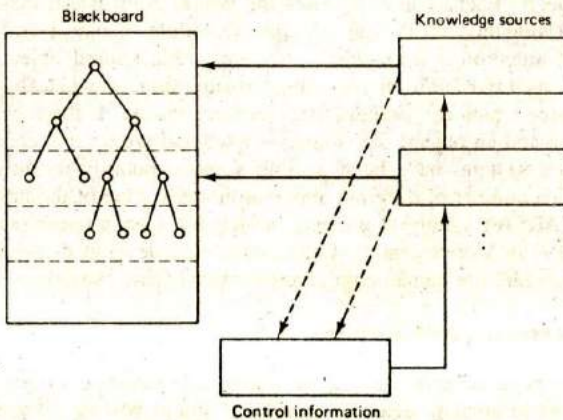


Figure 15.6 Components of blackboard systems.

part of a blackboard system.) The control knowledge monitors the changes to the blackboard and determines what the immediate focus of attention should be in solving the problem.

H. Penny Nii (1986a) has aptly described the blackboard problem solving strategy through the following analogy.

Imagine a room with a large blackboard on which a group of experts are piecing together a jigsaw puzzle. Each of the experts has some special knowledge about solving puzzles (e.g., a border expert, a shape expert, a color expert, etc.). Each member examines his or her pieces and decides if they will fit into the partially completed puzzle. Those members having appropriate pieces go up to the blackboard and update the evolving solution. The whole puzzle can be solved in complete silence with no direct communication among members of the group. Each person is self-activating, knowing when he or she can contribute to the solution. The solution evolves in this incremental way with each expert contributing dynamically on an opportunistic basis, that is, as the opportunity to contribute to the solution arises.

The objects on the blackboard are hierarchically organized into levels which facilitate analysis and solution. Information from one level serves as input to a set of knowledge sources. The sources modify the knowledge and place it on the same or different levels.

The control information is used by the control module to determine the focus of attention. This determines the next item to be processed. The focus of attention can be the choice of knowledge sources or the blackboard objects or both. If both, the control determines which sources to apply to which objects.

Problem solving proceeds with a knowledge source making changes to the blackboard objects. Each source indicates the contribution it can make to the new solution state. Using this information, the control module chooses a focus of attention. If the focus of attention is a knowledge source, a blackboard object is chosen as the context of its invocation. If the focus of attention is a blackboard object, a knowledge source which can process that object is chosen. If the focus of attention is both a source and an object, that source is executed within that context.

Blackboard systems have been gaining some popularity recently. They have been applied to a number of different application areas. One of the first applications was in the HEARSAY family of projects, which are speech understanding systems (Reddy et al., 1976). More recently, systems have been developed to analyze complex scenes, and to model the human cognitive processes (Nii, 1986b).

Analogical Reasoning Architectures

Little work has been done in the area of analogical reasoning systems. Yet this is one of the most promising areas for general problem solving. We humans make extensive use of our previous experience in solving everyday problems. This is because new problems are frequently similar to previously encountered problems.

Expert systems based on analogical architectures solve new problems like humans, by finding a similar problem solution that is known and applying the known solution to the new problem, possibly with some modifications. For example, if we know a method of proving that the product of two even integers is even, we can successfully prove that the product of two odd integers is odd through much the same proof steps. Only a slight modification will be required when collecting product terms in the result. Expert systems using analogical architectures will require a large knowledge base having numerous problem solutions and other previously-encountered situations or episodes. Each such situation should be stored as a unit in memory and be content-indexed for rapid retrieval. The inference mechanism must be able to extend known situations or solutions to fit the current problem and verify that the extended solution is reasonable. The author and one of his students has built a small toy analogical expert system in LISP to demonstrate many of the features needed for such systems (Patterson and Chu, 1988).

Neural Network Architectures

Neural networks are large networks of simple processing elements or nodes which process information dynamically in response to external inputs. The nodes are simplified models of neurons. The knowledge in a neural network is distributed throughout the network in the form of internode connections and weighted links which form the inputs to the nodes. The link weights serve to enhance or inhibit the input stimuli values which are then added together at the nodes. If the sum of all the inputs to a node exceeds some threshold value T , the node executes and produces an output which is passed on to other nodes or is used to produce some output response. In the simplest case, no output is produced if the total input is less than T . In more complex models, the output will depend on a nonlinear activation function.

Neural networks were originally inspired as being models of the human nervous system. They are greatly simplified models to be sure (neurons are known to be fairly complex processors). Even so, they have been shown to exhibit many "intelligent" abilities, such as learning, generalization, and abstraction.

A single node is illustrated in Figure 15.7. The inputs to the node are the values $x_1, x_2, \dots, x_n$, which typically take on values of $-1, 0, 1$, or real values within the range $(-1, 1)$. The weights $w_1, w_2, \dots, w_n$, correspond to the synaptic strengths of a neuron. They serve to increase or decrease the effects of the corresponding x_i input values. The sum of the products $x_i \cdot w_i, i = 1, 2, \dots, n$, serve as the total combined input to the node. If this sum is large enough to exceed the

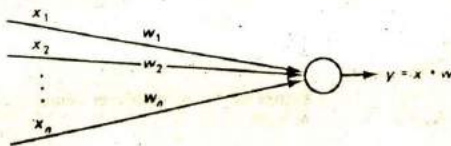


Figure 15.7 Model of a single neuron (node).

threshold amount T , the node fires, and produces an output y , an activation function value placed on the node's output links. This output may then be the input to other nodes or the final output response from the network.

Figure 15.8 illustrates three layers of a number of interconnected nodes. The first layer serves as the input layer, receiving inputs from some set of stimuli. The second layer (called the hidden layer) receives inputs from the first layer and produces a pattern of inputs to the third layer, the output layer. The pattern of outputs from the final layer are the network's responses to the input stimuli patterns. Input links to layer j ($j = 1, 2, 3$) have weights w_{ij} for $i = 1, 2, \dots, n$.

General multilayer networks having n nodes (number of rows) in each of m layers (number of columns of nodes) will have weights represented as an $n \times m$ matrix W . Using this representation, nodes having no interconnecting links will have a weight value of zero. Networks consisting of more than three layers would, of course, be correspondingly more complex than the network depicted in Figure 15.8.

A neural network can be thought of as a black box that transforms the input vector x to the output vector y where the transformation performed is the result of the pattern of connections and weights, that is, according to the values of the weight matrix W .

Consider the vector product

$$x * w = \sum x_i w_i$$

There is a geometric interpretation for this product. It is equivalent to projecting one vector onto the other vector in n -dimensional space. This notion is depicted in Figure 15.9 for the two-dimensional case.

The magnitude of the resultant vector is given by

$$x * w = |x||w| \cos \theta$$

where $|x|$ denotes the norm or length of the vector x . Note that this product is maximum when both vectors point in the same direction, that is, when $\theta = 0$. The

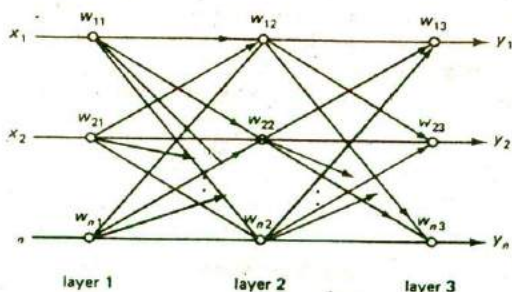


Figure 15.8 A multilayer neural network

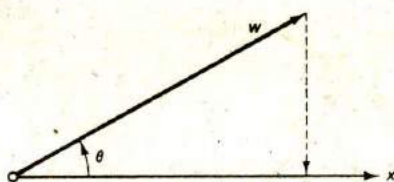


Figure 15.9 Vector multiplication is like vector projection.

product is a minimum when both point in opposite directions or when $\theta = 180$ degrees. This illustrates how the vectors in the weight matrix W influence the inputs to the nodes in a neural network.

Learning pattern weights. The interconnections and weights W in the neural network store the knowledge possessed by the network. These weights must be preset or learned in some manner. When learning is used, the process may be either supervised or unsupervised. In the supervised case, learning is performed by repeatedly presenting the network with an input pattern and a desired output response. The training examples then consist of the vector pairs (x, y') , where x is the input pattern and y' is the desired output response pattern. The weights are then adjusted until the difference between the actual output response y and the desired response y' are the same, that is until $D = y - y'$ is near zero.

One of the simpler supervised learning algorithms uses the following formula to adjust the weights W .

$$W_{\text{new}} = W_{\text{old}} + a * D * \frac{x}{|x|^2}$$

where $0 < a < 1$ is a learning constant that determines the rate of learning. When the difference D is large, the adjustment to the weights W is large, but when the output response y is close to the target response y' the adjustment will be small. When the difference D is near zero, the training process terminates at which point the network will produce the correct response for the given input patterns x .

In unsupervised learning, the training examples consist of the input vectors x only. No desired response y' is available to guide the system. Instead, the learning process must find the weights w_{ij} with no knowledge of the desired output response. We leave the unsupervised learning description until Chapter 18 where learning is covered in somewhat more detail.

A neural net expert system. An example of a simple neural network diagnostic expert system has been described by Stephen Gallant (1988). This system diagnoses and recommends treatments for acute sarcophagal disease. The system is illustrated in Figure 15.10. From six symptom variables $u_1, u_2, \dots, u_6$, one of two possible diseases can be diagnosed, u_7 or u_8 . From the resultant diagnosis, one of three treatments, u_9, u_{10} , or u_{11} can then be recommended.

When a given symptom is present, the corresponding variable is given a value